

# Northeast enrollment modeling

AUTHOR

John Baker

```
rm(list = ls())
```

```
library(tidyverse)
```

— Attaching core tidyverse packages — tidyverse 2.0.0 —

|             |       |           |       |
|-------------|-------|-----------|-------|
| ✓ dplyr     | 1.2.1 | ✓ readr   | 2.2.0 |
| ✓ forcats   | 1.0.1 | ✓ stringr | 1.6.0 |
| ✓ ggplot2   | 4.0.3 | ✓ tibble  | 3.3.1 |
| ✓ lubridate | 1.9.5 | ✓ tidyr   | 1.3.2 |
| ✓ purrr     | 1.2.2 |           |       |

— Conflicts — tidyverse\_conflicts() —

✖ dplyr::filter() masks stats::filter()

✖ dplyr::lag() masks stats::lag()

ℹ Use the conflicted package (<<http://conflicted.r-lib.org/>>) to force all conflicts to become errors

```
library(tidymodels)
```

— Attaching packages — tidymodels 1.5.0 —

|             |        |                |       |
|-------------|--------|----------------|-------|
| ✓ broom     | 1.0.13 | ✓ rsample      | 1.3.2 |
| ✓ dials     | 1.4.3  | ✓ tailor       | 0.1.0 |
| ✓ infer     | 1.1.0  | ✓ tune         | 2.1.0 |
| ✓ modeldata | 1.5.1  | ✓ workflows    | 1.3.0 |
| ✓ parsnip   | 1.6.0  | ✓ workflowsets | 1.1.1 |
| ✓ recipes   | 1.3.3  | ✓ yardstick    | 1.4.0 |

— Conflicts — tidymodels\_conflicts() —

✖ scales::discard() masks purrr::discard()

✖ dplyr::filter() masks stats::filter()

✖ recipes::fixed() masks stringr::fixed()

✖ dplyr::lag() masks stats::lag()

✖ yardstick::spec() masks readr::spec()

✖ recipes::step() masks stats::step()

```
library(leaps)
```

```
library(car)
```

Loading required package: carData

Attaching package: 'car'

The following object is masked from 'package:dplyr':

recode

The following object is masked from 'package:purrr':

some

```
library(doParallel)
```

Loading required package: foreach

Attaching package: 'foreach'

The following objects are masked from 'package:purrr':

accumulate, when

Loading required package: iterators

Loading required package: parallel

adding this to speed up code (from AI, code was taking forever)

```
cores = max(1, parallel::detectCores() - 1)
cl = parallel::makePSOCKcluster(cores)
registerDoParallel(cl)
```

```
ne = read_csv("northeast_census_over_time_modeling_gender.csv",
col_types = cols(ZIP = col_character(), .default = col_guess())) |>
  mutate(state = as.factor(state))
```

```
set.seed(88)
```

```
train_data = ne |> filter(year < 2024)
test_data = ne |> filter(year == 2024)
```

```
set.seed(88)
```

```
folds_grouped = group_vfold_cv(train_data, group = ZIP, v = 10)
```

Before doing recipes, let's do some subset work

```
subset_data = train_data |>
  select(
    private_k12,
    # Economic characteristics
    median_hh_income_inflation_adjusted,
    median_home_value_inflation_adjusted,
    # Adult educational attainment
    pct_adults_hs_plus,
    pct_adults_bachelors_plus,
    pct_adults_grad_plus,
    # Household characteristics
```

```

total_households,
pct_households_with_children,
# Child population by age
age_0_4,
age_5_9,
age_10_14,
age_15_17,
# Gender composition
pct_school_age_male)

subset_fit = regsubsets(
  private_k12 ~ .,
  data = subset_data,
  nvmax = 12,
  method = "exhaustive")

subset_summary = summary(subset_fit)
subset_comparison = tibble(
  predictors = 1:12,
  adjusted_r2 = subset_summary$adjr2,
  cp = subset_summary$cp,
  bic = subset_summary$bic)

subset_comparison

```

```

# A tibble: 12 × 4
  predictors adjusted_r2    cp    bic
      <int>      <dbl> <dbl> <dbl>
1         1      0.465 6742. -18812.
2         2      0.517 3179. -21865.
3         3      0.536 1882. -23053.
4         4      0.545 1258. -23638.
5         5      0.552  770. -24102.
6         6      0.559  328. -24528.
7         7      0.560  221. -24626.
8         8      0.561  152. -24686.
9         9      0.562   79.8 -24749.
10        10      0.563   28.6 -24792.
11        11      0.563   11.4 -24801.
12        12      0.563   13.0 -24791.

```

Which works best?

```

best_adjr2 = which.max(subset_summary$adjr2)
best_cp = which.min(subset_summary$cp)
best_bic = which.min(subset_summary$bic)

best_adjr2

```

[1] 11

```
best_cp
```

```
[1] 11
```

```
best_bic
```

```
[1] 11
```

```
coef(subset_fit, id = best_adjr2)
```

|                                      |                                     |
|--------------------------------------|-------------------------------------|
| (Intercept)                          | median_hh_income_inflation_adjusted |
| -6.848590e+02                        | -1.252612e-03                       |
| median_home_value_inflation_adjusted | pct_adults_hs_plus                  |
| 3.127957e-04                         | 1.085606e+03                        |
| pct_adults_bachelors_plus            | pct_adults_grad_plus                |
| -2.243203e+02                        | 7.007451e+02                        |
| total_households                     | pct_households_with_children        |
| -8.782474e-02                        | -9.305260e+02                       |
| age_0_4                              | age_5_9                             |
| 6.435522e-01                         | 2.745301e-01                        |
| age_10_14                            | age_15_17                           |
| 1.009725e-01                         | -2.111701e-01                       |

```
coef(subset_fit, id = best_cp)
```

|                                      |                                     |
|--------------------------------------|-------------------------------------|
| (Intercept)                          | median_hh_income_inflation_adjusted |
| -6.848590e+02                        | -1.252612e-03                       |
| median_home_value_inflation_adjusted | pct_adults_hs_plus                  |
| 3.127957e-04                         | 1.085606e+03                        |
| pct_adults_bachelors_plus            | pct_adults_grad_plus                |
| -2.243203e+02                        | 7.007451e+02                        |
| total_households                     | pct_households_with_children        |
| -8.782474e-02                        | -9.305260e+02                       |
| age_0_4                              | age_5_9                             |
| 6.435522e-01                         | 2.745301e-01                        |
| age_10_14                            | age_15_17                           |
| 1.009725e-01                         | -2.111701e-01                       |

```
coef(subset_fit, id = best_bic)
```

|                                      |                                     |
|--------------------------------------|-------------------------------------|
| (Intercept)                          | median_hh_income_inflation_adjusted |
| -6.848590e+02                        | -1.252612e-03                       |
| median_home_value_inflation_adjusted | pct_adults_hs_plus                  |
| 3.127957e-04                         | 1.085606e+03                        |
| pct_adults_bachelors_plus            | pct_adults_grad_plus                |
| -2.243203e+02                        | 7.007451e+02                        |
| total_households                     | pct_households_with_children        |
| -8.782474e-02                        | -9.305260e+02                       |
| age_0_4                              | age_5_9                             |
| 6.435522e-01                         | 2.745301e-01                        |

age\_10\_14  
1.009725e-01

age\_15\_17  
-2.111701e-01

Looks like 11! Will try three models to be sure.

```
# Recipe A: All features
```

```
recipe_a = recipe(  
  private_k12 ~ median_hh_income_inflation_adjusted + median_home_value_inflation_adjusted  
    +  
    school_age_population + pct_adults_hs_plus + pct_adults_bachelors_plus +  
    pct_adults_grad_plus + total_households + pct_households_with_children + age_0_4 +  
    state,  
  data = train_data) |>  
  step_dummy(all_nominal_predictors()) |>  
  step_zv(all_predictors())
```

```
# Recipe B: 11 numeric predictors + state, no gender
```

```
recipe_b = recipe(  
  private_k12 ~ median_hh_income_inflation_adjusted + median_home_value_inflation_adjusted  
    +  
    pct_adults_hs_plus + pct_adults_bachelors_plus + pct_adults_grad_plus +  
    total_households +  
    pct_households_with_children + age_0_4 + age_5_9 + age_10_14 + age_15_17 + state,  
  data = train_data) |>  
  step_dummy(all_nominal_predictors()) |>  
  step_zv(all_predictors())
```

```
# Recipe C: Same as B, no bachelors for parents
```

```
recipe_c = recipe(  
  private_k12 ~ median_hh_income_inflation_adjusted + median_home_value_inflation_adjusted  
    +  
    pct_adults_hs_plus + pct_adults_grad_plus + total_households +  
    pct_households_with_children + age_0_4 + age_5_9 + age_10_14 + age_15_17 + state,  
  data = train_data) |>  
  step_dummy(all_nominal_predictors()) |>  
  step_zv(all_predictors())
```

```
missing_a = prep(recipe_a) |>  
  bake(new_data = NULL) |>  
  summarize(across(everything(), ~sum(is.na(.x))))
```

```
missing_b = prep(recipe_b) |>  
  bake(new_data = NULL) |>  
  summarize(across(everything(), ~sum(is.na(.x))))
```

```
missing_c = prep(recipe_c) |>  
  bake(new_data = NULL) |>  
  summarize(across(everything(), ~sum(is.na(.x))))
```

### missing\_a

```
# A tibble: 1 × 23
  median_hh_income_inflation_adju...1 median_home_value_inf...2 school_age_population
                                <int>                                <int>                                <int>
1                                0                                0                                0
# i abbreviated names: 1median_hh_income_inflation_adjusted,
# 2median_home_value_inflation_adjusted
# i 20 more variables: pct_adults_hs_plus <int>,
# pct_adults_bachelors_plus <int>, pct_adults_grad_plus <int>,
# total_households <int>, pct_households_with_children <int>, age_0_4 <int>,
# private_k12 <int>, state_DC <int>, state_DE <int>, state_MA <int>,
# state_MD <int>, state_ME <int>, state_NH <int>, state_NJ <int>, ...
```

### missing\_b

```
# A tibble: 1 × 25
  median_hh_income_inflation_adjusted median_home_value_inf...1 pct_adults_hs_plus
                                <int>                                <int>                                <int>
1                                0                                0                                0
# i abbreviated name: 1median_home_value_inflation_adjusted
# i 22 more variables: pct_adults_bachelors_plus <int>,
# pct_adults_grad_plus <int>, total_households <int>,
# pct_households_with_children <int>, age_0_4 <int>, age_5_9 <int>,
# age_10_14 <int>, age_15_17 <int>, private_k12 <int>, state_DC <int>,
# state_DE <int>, state_MA <int>, state_MD <int>, state_ME <int>,
# state_NH <int>, state_NJ <int>, state_NY <int>, state_PA <int>, ...
```

### missing\_c

```
# A tibble: 1 × 24
  median_hh_income_inflation_adjusted median_home_value_inf...1 pct_adults_hs_plus
                                <int>                                <int>                                <int>
1                                0                                0                                0
# i abbreviated name: 1median_home_value_inflation_adjusted
# i 21 more variables: pct_adults_grad_plus <int>, total_households <int>,
# pct_households_with_children <int>, age_0_4 <int>, age_5_9 <int>,
# age_10_14 <int>, age_15_17 <int>, private_k12 <int>, state_DC <int>,
# state_DE <int>, state_MA <int>, state_MD <int>, state_ME <int>,
# state_NH <int>, state_NJ <int>, state_NY <int>, state_PA <int>,
# state_RI <int>, state_VA <int>, state_VT <int>, state_WV <int>
```

```
reg_metrics = metric_set(rmse, rsq, mae)
reg_metrics
```

A metric set, consisting of:

- `rmse()`, a numeric metric | direction: minimize
- `rsq()`, a numeric metric | direction: maximize
- `mae()`, a numeric metric | direction: minimize

```

set.seed(88)
lr_spec = linear_reg(mode = "regression") |> set_engine("lm")

lr_wf_a = workflow() |> add_recipe(recipe_a) |> add_model(lr_spec)
lr_wf_b = workflow() |> add_recipe(recipe_b) |> add_model(lr_spec)
lr_wf_c = workflow() |> add_recipe(recipe_c) |> add_model(lr_spec)

lr_cv_a = fit_resamples(lr_wf_a,
  resamples = folds_grouped,
  metrics = reg_metrics,
  control = control_resamples(save_pred = TRUE))
lr_cv_b = fit_resamples(lr_wf_b,
  resamples = folds_grouped,
  metrics = reg_metrics,
  control = control_resamples(save_pred = TRUE))
lr_cv_c = fit_resamples(lr_wf_c,
  resamples = folds_grouped,
  metrics = reg_metrics,
  control = control_resamples(save_pred = TRUE))

```

```

linreg_metrics = bind_rows(
  collect_metrics(lr_cv_a) |> mutate(model = "OLS A - Aggregate ages"),
  collect_metrics(lr_cv_b) |> mutate(model = "OLS B - Full individual ages"),
  collect_metrics(lr_cv_c) |> mutate(model = "OLS C - Reduced individual ages")) |>
  dplyr::select(model, .metric, mean, std_err) |>
  arrange(.metric, mean)

```

```
linreg_metrics
```

# A tibble: 9 × 4

|   | model<br><chr>                  | .metric<br><chr> | mean<br><dbl> | std_err<br><dbl> |
|---|---------------------------------|------------------|---------------|------------------|
| 1 | OLS A - Aggregate ages          | mae              | 186.          | 3.31             |
| 2 | OLS C - Reduced individual ages | mae              | 187.          | 3.19             |
| 3 | OLS B - Full individual ages    | mae              | 187.          | 3.19             |
| 4 | OLS A - Aggregate ages          | rmse             | 425.          | 47.2             |
| 5 | OLS C - Reduced individual ages | rmse             | 425.          | 46.4             |
| 6 | OLS B - Full individual ages    | rmse             | 425.          | 46.4             |
| 7 | OLS B - Full individual ages    | rsq              | 0.531         | 0.0383           |
| 8 | OLS C - Reduced individual ages | rsq              | 0.531         | 0.0384           |
| 9 | OLS A - Aggregate ages          | rsq              | 0.533         | 0.0371           |

```

ols_fit_a = lr_wf_a |> fit(data = train_data)
ols_fit_b = lr_wf_b |> fit(data = train_data)
ols_fit_c = lr_wf_c |> fit(data = train_data)
ols_fit_a |> extract_fit_engine() |> summary()

```

Call:

```
stats::lm(formula = ..y ~ ., data = data)
```

Residuals:

| Min     | 1Q     | Median | 3Q    | Max     |
|---------|--------|--------|-------|---------|
| -4853.7 | -106.2 | -3.3   | 102.3 | 22146.8 |

Coefficients:

|                                      | Estimate   | Std. Error | t value | Pr(> t ) |     |
|--------------------------------------|------------|------------|---------|----------|-----|
| (Intercept)                          | -7.190e+02 | 4.541e+01  | -15.831 | < 2e-16  | *** |
| median_hh_income_inflation_adjusted  | -1.458e-03 | 1.421e-04  | -10.262 | < 2e-16  | *** |
| median_home_value_inflation_adjusted | 3.717e-04  | 1.906e-05  | 19.498  | < 2e-16  | *** |
| school_age_population                | 6.948e-02  | 4.080e-03  | 17.028  | < 2e-16  | *** |
| pct_adults_hs_plus                   | 1.043e+03  | 5.106e+01  | 20.426  | < 2e-16  | *** |
| pct_adults_bachelors_plus            | -6.257e+01 | 5.379e+01  | -1.163  | 0.244724 |     |
| pct_adults_grad_plus                 | 5.881e+02  | 8.087e+01  | 7.273   | 3.60e-13 | *** |
| total_households                     | -9.299e-02 | 1.304e-03  | -71.293 | < 2e-16  | *** |
| pct_households_with_children         | -9.489e+02 | 3.919e+01  | -24.212 | < 2e-16  | *** |
| age_0_4                              | 7.384e-01  | 8.924e-03  | 82.745  | < 2e-16  | *** |
| state_DC                             | -2.300e+02 | 4.644e+01  | -4.951  | 7.41e-07 | *** |
| state_DE                             | 5.574e+01  | 2.850e+01  | 1.956   | 0.050461 | .   |
| state_MA                             | -5.598e+01 | 1.481e+01  | -3.780  | 0.000157 | *** |
| state_MD                             | 3.754e+01  | 1.554e+01  | 2.415   | 0.015742 | *   |
| state_ME                             | 1.960e+01  | 1.660e+01  | 1.181   | 0.237667 |     |
| state_NH                             | 1.756e+01  | 1.786e+01  | 0.983   | 0.325387 |     |
| state_NJ                             | -3.001e+01 | 1.456e+01  | -2.062  | 0.039226 | *   |
| state_NY                             | 3.893e+01  | 1.305e+01  | 2.983   | 0.002860 | **  |
| state_PA                             | 1.165e+02  | 1.327e+01  | 8.782   | < 2e-16  | *** |
| state_RI                             | 9.227e+01  | 2.607e+01  | 3.539   | 0.000403 | *** |
| state_VA                             | 1.114e+01  | 1.409e+01  | 0.791   | 0.429168 |     |
| state_VT                             | -9.465e+00 | 1.909e+01  | -0.496  | 0.620039 |     |
| state_WV                             | 1.277e+02  | 1.605e+01  | 7.955   | 1.86e-15 | *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 418.9 on 30054 degrees of freedom

Multiple R-squared: 0.5651, Adjusted R-squared: 0.5648

F-statistic: 1775 on 22 and 30054 DF, p-value: < 2.2e-16

```
ols_fit_b |> extract_fit_engine() |> summary()
```

Call:

```
stats::lm(formula = ..y ~ ., data = data)
```

Residuals:

| Min     | 1Q     | Median | 3Q    | Max     |
|---------|--------|--------|-------|---------|
| -4871.5 | -106.4 | -4.4   | 103.4 | 21688.7 |

Coefficients:

|                                      | Estimate   | Std. Error | t value | Pr(> t ) |     |
|--------------------------------------|------------|------------|---------|----------|-----|
| (Intercept)                          | -7.200e+02 | 4.515e+01  | -15.948 | < 2e-16  | *** |
| median_hh_income_inflation_adjusted  | -1.285e-03 | 1.416e-04  | -9.077  | < 2e-16  | *** |
| median_home_value_inflation_adjusted | 3.645e-04  | 1.896e-05  | 19.220  | < 2e-16  | *** |
| pct_adults_hs_plus                   | 1.041e+03  | 5.076e+01  | 20.505  | < 2e-16  | *** |

|                              |            |           |         |          |     |
|------------------------------|------------|-----------|---------|----------|-----|
| pct_adults_bachelors_plus    | -7.014e+01 | 5.348e+01 | -1.312  | 0.189691 |     |
| pct_adults_grad_plus         | 5.542e+02  | 8.041e+01 | 6.892   | 5.60e-12 | *** |
| total_households             | -8.876e-02 | 1.316e-03 | -67.441 | < 2e-16  | *** |
| pct_households_with_children | -9.336e+02 | 3.897e+01 | -23.955 | < 2e-16  | *** |
| age_0_4                      | 6.495e-01  | 1.021e-02 | 63.586  | < 2e-16  | *** |
| age_5_9                      | 2.750e-01  | 1.462e-02 | 18.807  | < 2e-16  | *** |
| age_10_14                    | 9.767e-02  | 1.394e-02 | 7.009   | 2.46e-12 | *** |
| age_15_17                    | -2.069e-01 | 1.862e-02 | -11.108 | < 2e-16  | *** |
| state_DC                     | -2.656e+02 | 4.622e+01 | -5.747  | 9.20e-09 | *** |
| state_DE                     | 4.550e+01  | 2.833e+01 | 1.606   | 0.108362 |     |
| state_MA                     | -5.657e+01 | 1.472e+01 | -3.842  | 0.000122 | *** |
| state_MD                     | 2.669e+01  | 1.546e+01 | 1.726   | 0.084338 | .   |
| state_ME                     | 1.283e+01  | 1.651e+01 | 0.777   | 0.436986 |     |
| state_NH                     | 1.126e+01  | 1.776e+01 | 0.634   | 0.526065 |     |
| state_NJ                     | -3.578e+01 | 1.447e+01 | -2.472  | 0.013450 | *   |
| state_NY                     | 3.628e+01  | 1.298e+01 | 2.796   | 0.005172 | **  |
| state_PA                     | 1.095e+02  | 1.319e+01 | 8.295   | < 2e-16  | *** |
| state_RI                     | 9.013e+01  | 2.592e+01 | 3.477   | 0.000508 | *** |
| state_VA                     | 3.681e-01  | 1.402e+01 | 0.026   | 0.979057 |     |
| state_VT                     | -1.473e+01 | 1.898e+01 | -0.776  | 0.437684 |     |
| state_WV                     | 1.189e+02  | 1.596e+01 | 7.449   | 9.66e-14 | *** |

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 416.4 on 30052 degrees of freedom  
Multiple R-squared: 0.5702, Adjusted R-squared: 0.5699  
F-statistic: 1661 on 24 and 30052 DF, p-value: < 2.2e-16

```
ols_fit_c |> extract_fit_engine() |> summary()
```

Call:

```
stats::lm(formula = .y ~ ., data = data)
```

Residuals:

|         |        |        |       |         |
|---------|--------|--------|-------|---------|
| Min     | 1Q     | Median | 3Q    | Max     |
| -4871.2 | -106.5 | -4.2   | 103.3 | 21688.6 |

Coefficients:

|                                      | Estimate   | Std. Error | t value | Pr(> t ) |     |
|--------------------------------------|------------|------------|---------|----------|-----|
| (Intercept)                          | -7.062e+02 | 4.390e+01  | -16.087 | < 2e-16  | *** |
| median_hh_income_inflation_adjusted  | -1.326e-03 | 1.380e-04  | -9.608  | < 2e-16  | *** |
| median_home_value_inflation_adjusted | 3.603e-04  | 1.869e-05  | 19.275  | < 2e-16  | *** |
| pct_adults_hs_plus                   | 1.020e+03  | 4.808e+01  | 21.203  | < 2e-16  | *** |
| pct_adults_grad_plus                 | 4.657e+02  | 4.367e+01  | 10.663  | < 2e-16  | *** |
| total_households                     | -8.903e-02 | 1.300e-03  | -68.482 | < 2e-16  | *** |
| pct_households_with_children         | -9.316e+02 | 3.895e+01  | -23.922 | < 2e-16  | *** |
| age_0_4                              | 6.492e-01  | 1.021e-02  | 63.572  | < 2e-16  | *** |
| age_5_9                              | 2.752e-01  | 1.462e-02  | 18.819  | < 2e-16  | *** |
| age_10_14                            | 9.813e-02  | 1.393e-02  | 7.044   | 1.91e-12 | *** |
| age_15_17                            | -2.065e-01 | 1.862e-02  | -11.088 | < 2e-16  | *** |
| state_DC                             | -2.601e+02 | 4.603e+01  | -5.651  | 1.61e-08 | *** |

|          |            |           |        |          |     |
|----------|------------|-----------|--------|----------|-----|
| state_DE | 4.597e+01  | 2.833e+01 | 1.622  | 0.104747 |     |
| state_MA | -5.696e+01 | 1.472e+01 | -3.870 | 0.000109 | *** |
| state_MD | 2.800e+01  | 1.543e+01 | 1.814  | 0.069638 | .   |
| state_ME | 1.188e+01  | 1.649e+01 | 0.720  | 0.471359 |     |
| state_NH | 9.929e+00  | 1.773e+01 | 0.560  | 0.575465 |     |
| state_NJ | -3.753e+01 | 1.441e+01 | -2.604 | 0.009224 | **  |
| state_NY | 3.746e+01  | 1.294e+01 | 2.894  | 0.003804 | **  |
| state_PA | 1.099e+02  | 1.319e+01 | 8.334  | < 2e-16  | *** |
| state_RI | 8.896e+01  | 2.591e+01 | 3.434  | 0.000596 | *** |
| state_VA | 2.649e-01  | 1.402e+01 | 0.019  | 0.984927 |     |
| state_VT | -1.688e+01 | 1.891e+01 | -0.893 | 0.371918 |     |
| state_WV | 1.199e+02  | 1.594e+01 | 7.521  | 5.60e-14 | *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 416.4 on 30053 degrees of freedom  
Multiple R-squared: 0.5702, Adjusted R-squared: 0.5698  
F-statistic: 1733 on 23 and 30053 DF, p-value: < 2.2e-16

I'm curious about outliers.

```
extreme_five = c(
  "08701", # Lakewood, NJ
  "11219", # Brooklyn, NY
  "10977", # Spring Valley, NY
  "10950", # Monroe, NY
  "10952") # Monsey, NY

midatlantic_high = test_data |>
  filter(state %in% c("DC", "MD", "VA")) |>
  arrange(desc(private_k12)) |>
  group_by(state) |>
  slice_head(n = 3) |>
  ungroup() |>
  dplyr::select(ZIP, zip_place_name, state, private_k12, school_age_population)
```

midatlantic\_high

# A tibble: 9 × 5

|   | ZIP   | zip_place_name | state | private_k12 | school_age_population |
|---|-------|----------------|-------|-------------|-----------------------|
|   | <chr> | <chr>          | <fct> | <dbl>       | <dbl>                 |
| 1 | 20016 | Washington     | DC    | 2484        | 5057                  |
| 2 | 20011 | Washington     | DC    | 1747        | 12289                 |
| 3 | 20032 | Washington     | DC    | 1376        | 7662                  |
| 4 | 21209 | Baltimore      | MD    | 3399        | 6353                  |
| 5 | 20854 | Potomac        | MD    | 2617        | 9945                  |
| 6 | 21215 | Baltimore      | MD    | 2571        | 8350                  |
| 7 | 22101 | Mc Lean        | VA    | 1881        | 6694                  |
| 8 | 22207 | Arlington      | VA    | 1745        | 7860                  |
| 9 | 23434 | Suffolk        | VA    | 1704        | 9386                  |

```

diagnostic_zips = c(extreme_five, midatlantic_high$ZIP) |> unique()
train_lookup = train_data |>
  mutate(.row = row_number()) |>
  dplyr::select(.row, ZIP, year, state, zip_place_name)

pred_a = collect_predictions(lr_cv_a) |>
  left_join(train_lookup, by = ".row") |>
  mutate(model = "A - Aggregate ages")
pred_b = collect_predictions(lr_cv_b) |>
  left_join(train_lookup, by = ".row") |>
  mutate(model = "B - Full individual ages")
pred_c = collect_predictions(lr_cv_c) |>
  left_join(train_lookup, by = ".row") |>
  mutate(model = "C - Reduced individual ages")

all_ols_predictions = bind_rows(pred_a, pred_b, pred_c)
all_ols_predictions = all_ols_predictions |>
  mutate(market_group = case_when(
    ZIP %in% extreme_five ~ "Original five extreme ZIPs",
    ZIP %in% diagnostic_zips ~ "High DC / MD / VA ZIPs",
    TRUE ~ "Other ZIPs"))

extreme_comparison = all_ols_predictions |>
  group_by(model, market_group) |>
  summarize(
    n = n(),
    mae = mae_vec(truth = private_k12, estimate = .pred),
    rmse = rmse_vec(truth = private_k12, estimate = .pred),
    mean_error = mean(.pred - private_k12),
    .groups = "drop")

extreme_comparison

```

# A tibble: 9 × 6

|   | model<br><chr>              | market_group<br><chr>  | n<br><int> | mae<br><dbl> | rmse<br><dbl> | mean_error<br><dbl> |
|---|-----------------------------|------------------------|------------|--------------|---------------|---------------------|
| 1 | A - Aggregate ages          | High DC / MD / VA Z... | 45         | 877.         | 1016.         | -393.               |
| 2 | A - Aggregate ages          | Original five extre... | 25         | 9202.        | 10471.        | -9202.              |
| 3 | A - Aggregate ages          | Other ZIPs             | 30007      | 177.         | 328.          | 7.17                |
| 4 | B - Full individual ages    | High DC / MD / VA Z... | 45         | 911.         | 1054.         | -381.               |
| 5 | B - Full individual ages    | Original five extre... | 25         | 9155.        | 10411.        | -9155.              |
| 6 | B - Full individual ages    | Other ZIPs             | 30007      | 178.         | 329.          | 7.16                |
| 7 | C - Reduced individual ages | High DC / MD / VA Z... | 45         | 913.         | 1058.         | -386.               |
| 8 | C - Reduced individual ages | Original five extre... | 25         | 9154.        | 10411.        | -9154.              |
| 9 | C - Reduced individual ages | Other ZIPs             | 30007      | 178.         | 329.          | 7.15                |

```

zip_comparison = all_ols_predictions |>
  filter(ZIP %in% diagnostic_zips) |>
  group_by(model, ZIP, zip_place_name, state) |>
  summarize(
    actual_mean = mean(private_k12),
    predicted_mean = mean(.pred),

```

```

    mean_error = mean(.pred - private_k12),
    mae = mean(abs(.pred - private_k12)),
    .groups = "drop") |>
  arrange(ZIP, model)

```

zip\_comparison

```

# A tibble: 42 × 8
  model ZIP zip_place_name state actual_mean predicted_mean mean_error mae
  <chr> <chr> <chr>         <fct>         <dbl>         <dbl>         <dbl> <dbl>
1 A - ... 08701 Lakewood    NJ          27191.         9557.        -17635. 17635.
2 B - ... 08701 Lakewood    NJ          27191.         9728.        -17463. 17463.
3 C - ... 08701 Lakewood    NJ          27191.         9728.        -17464. 17464.
4 A - ... 10950 Monroe      NY          10797.         5045.         -5752.  5752.
5 B - ... 10950 Monroe      NY          10797.         5125.         -5672.  5672.
6 C - ... 10950 Monroe      NY          10797.         5126.         -5671.  5671.
7 A - ... 10952 Monsey      NY          12878.         4642.         -8235.  8235.
8 B - ... 10952 Monsey      NY          12878.         4542.         -8336.  8336.
9 C - ... 10952 Monsey      NY          12878.         4543.         -8335.  8335.
10 A - ... 10977 Spring Valley NY          11752.         5030.         -6722.  6722.
# i 32 more rows

```

So I have some concerns about multicollinearity here. Let's check!

```

vif_a = ols_fit_a |>
  extract_fit_engine() |>
  car::vif()

vif_c = ols_fit_c |>
  extract_fit_engine() |>
  car::vif()

# Compare VIF values
tibble(
  variable = names(vif_a),
  VIF = as.numeric(vif_a)) |>
  arrange(desc(VIF))

```

```

# A tibble: 22 × 2
  variable VIF
  <chr>    <dbl>
1 school_age_population 17.1
2 pct_adults_bachelors_plus 16.1
3 age_0_4 13.9
4 pct_adults_grad_plus 11.3
5 total_households 8.97
6 median_hh_income_inflation_adjusted 5.48
7 state_NY 5.08
8 state_PA 4.95
9 median_home_value_inflation_adjusted 3.78
10 state_VA 3.34
# i 12 more rows

```

```
tibble(
  variable = names(vif_c),
  VIF = as.numeric(vif_c)) |>
  arrange(desc(VIF))
```

# A tibble: 23 × 2

|                  | variable                             | VIF   |
|------------------|--------------------------------------|-------|
|                  | <chr>                                | <dbl> |
| 1                | age_5_9                              | 33.4  |
| 2                | age_10_14                            | 30.6  |
| 3                | age_15_17                            | 19.7  |
| 4                | age_0_4                              | 18.4  |
| 5                | total_households                     | 9.02  |
| 6                | median_hh_income_inflation_adjusted  | 5.23  |
| 7                | state_NY                             | 5.06  |
| 8                | state_PA                             | 4.95  |
| 9                | median_home_value_inflation_adjusted | 3.67  |
| 10               | pct_adults_grad_plus                 | 3.35  |
| # i 13 more rows |                                      |       |

Let's try an additional Model D that could improve on these high VIF values

```
set.seed(88)

recipe_d = recipe(
  private_k12 ~ median_hh_income_inflation_adjusted + median_home_value_inflation_adjusted
  +
  school_age_population + pct_adults_hs_plus + pct_adults_grad_plus +
  total_households + pct_households_with_children + state,
  data = train_data) |>
  step_dummy(all_nominal_predictors()) |>
  step_zv(all_predictors())

lr_wf_d = workflow() |>
  add_recipe(recipe_d) |>
  add_model(lr_spec)

lr_cv_d = fit_resamples(lr_wf_d,
  resamples = folds_grouped,
  metrics = reg_metrics,
  control = control_resamples(save_pred = TRUE))

ols_fit_d = lr_wf_d |>
  fit(data = train_data)

vif_d = ols_fit_d |>
  extract_fit_engine() |>
  car::vif()

tibble(
  variable = names(vif_d),
```

```
VIF = as.numeric(vif_d)) |>
  arrange(desc(VIF))
```

# A tibble: 20 × 2

|    | variable                             | VIF   |
|----|--------------------------------------|-------|
|    | <chr>                                | <dbl> |
| 1  | school_age_population                | 8.49  |
| 2  | total_households                     | 8.35  |
| 3  | state_NY                             | 5.04  |
| 4  | median_hh_income_inflation_adjusted  | 5.02  |
| 5  | state_PA                             | 4.94  |
| 6  | median_home_value_inflation_adjusted | 3.57  |
| 7  | pct_adults_grad_plus                 | 3.34  |
| 8  | state_VA                             | 3.33  |
| 9  | state_NJ                             | 2.89  |
| 10 | state_MA                             | 2.67  |
| 11 | state_WV                             | 2.48  |
| 12 | state_MD                             | 2.30  |
| 13 | state_ME                             | 2.19  |
| 14 | pct_adults_hs_plus                   | 1.97  |
| 15 | state_NH                             | 1.77  |
| 16 | pct_households_with_children         | 1.71  |
| 17 | state_VT                             | 1.69  |
| 18 | state_RI                             | 1.25  |
| 19 | state_DE                             | 1.21  |
| 20 | state_DC                             | 1.11  |

```
linreg_metrics_acd = bind_rows(
  collect_metrics(lr_cv_a) |>
    mutate(model = "OLS A - Aggregate ages"),

  collect_metrics(lr_cv_c) |>
    mutate(model = "OLS C - Reduced individual ages"),

  collect_metrics(lr_cv_d) |>
    mutate(model = "OLS D - Reduced aggregate ages")
) |>
  select(
    model,
    .metric,
    mean,
    std_err
  ) |>
  arrange(.metric, mean)

linreg_metrics_acd
```

# A tibble: 9 × 4

|   | model                          | .metric | mean  | std_err |
|---|--------------------------------|---------|-------|---------|
|   | <chr>                          | <chr>   | <dbl> | <dbl>   |
| 1 | OLS D - Reduced aggregate ages | mae     | 177.  | 3.09    |
| 2 | OLS A - Aggregate ages         | mae     | 186.  | 3.31    |

|   |                                 |      |       |        |
|---|---------------------------------|------|-------|--------|
| 3 | OLS C – Reduced individual ages | mae  | 187.  | 3.19   |
| 4 | OLS A – Aggregate ages          | rmse | 425.  | 47.2   |
| 5 | OLS C – Reduced individual ages | rmse | 425.  | 46.4   |
| 6 | OLS D – Reduced aggregate ages  | rmse | 450.  | 62.6   |
| 7 | OLS D – Reduced aggregate ages  | rsq  | 0.500 | 0.0256 |
| 8 | OLS C – Reduced individual ages | rsq  | 0.531 | 0.0384 |
| 9 | OLS A – Aggregate ages          | rsq  | 0.533 | 0.0371 |

Model D improves upon model A in MAE, but worse in RMSE. I'm curious to use A, C, and D in a regularized regression. We see outliers, especially in towns like Lakewood Township, NJ where a lot of Orthodox Jews live who are far less likely to attend public schools. Model D does not perform as well on the outliers but performs better on most ZIPs. This is more interesting to me, I'd rather it predict 99% right and be off on the outliers that don't follow other patterns.

```
recipe_a_reg = recipe_a |>
  step_normalize(all_numeric_predictors())
recipe_c_reg = recipe_c |>
  step_normalize(all_numeric_predictors())
recipe_d_reg = recipe_d |>
  step_normalize(all_numeric_predictors())

reg_spec = linear_reg(mode = "regression", penalty = tune(), mixture = tune()) |>
  set_engine("glmnet")

# workflows

reg_wf_a = workflow() |> add_recipe(recipe_a_reg) |> add_model(reg_spec)
reg_wf_c = workflow() |> add_recipe(recipe_c_reg) |> add_model(reg_spec)
reg_wf_d = workflow() |> add_recipe(recipe_d_reg) |> add_model(reg_spec)

# tuning grid

reg_grid = crossing(
  penalty = 10^seq(-4, 3, length.out = 20),
  mixture = c(0, 0.5, 1))
reg_grid
```

```
# A tibble: 60 × 2
  penalty mixture
  <dbl>     <dbl>
1 0.0001      0
2 0.0001     0.5
3 0.0001      1
4 0.000234    0
5 0.000234   0.5
6 0.000234    1
7 0.000546    0
8 0.000546   0.5
9 0.000546    1
10 0.00127     0
# i 50 more rows
```

```
set.seed(88)
```

```
reg_cv_a = tune_grid(  
  reg_wf_a,  
  resamples = folds_grouped,  
  grid = reg_grid,  
  metrics = reg_metrics,  
  control = control_grid())
```

→ A | warning: A correlation computation is required, but `estimate` is constant and has 0 standard deviation, resulting in a divide by 0 error. `NA` will be returned.

There were issues with some computations A: x2

There were issues with some computations A: x4

There were issues with some computations A: x16

There were issues with some computations A: x22

```
reg_cv_c = tune_grid(  
  reg_wf_c,  
  resamples = folds_grouped,  
  grid = reg_grid,  
  metrics = reg_metrics,  
  control = control_grid())
```

→ A | warning: A correlation computation is required, but `estimate` is constant and has 0 standard deviation, resulting in a divide by 0 error. `NA` will be returned.

There were issues with some computations A: x6

There were issues with some computations A: x20

There were issues with some computations A: x22

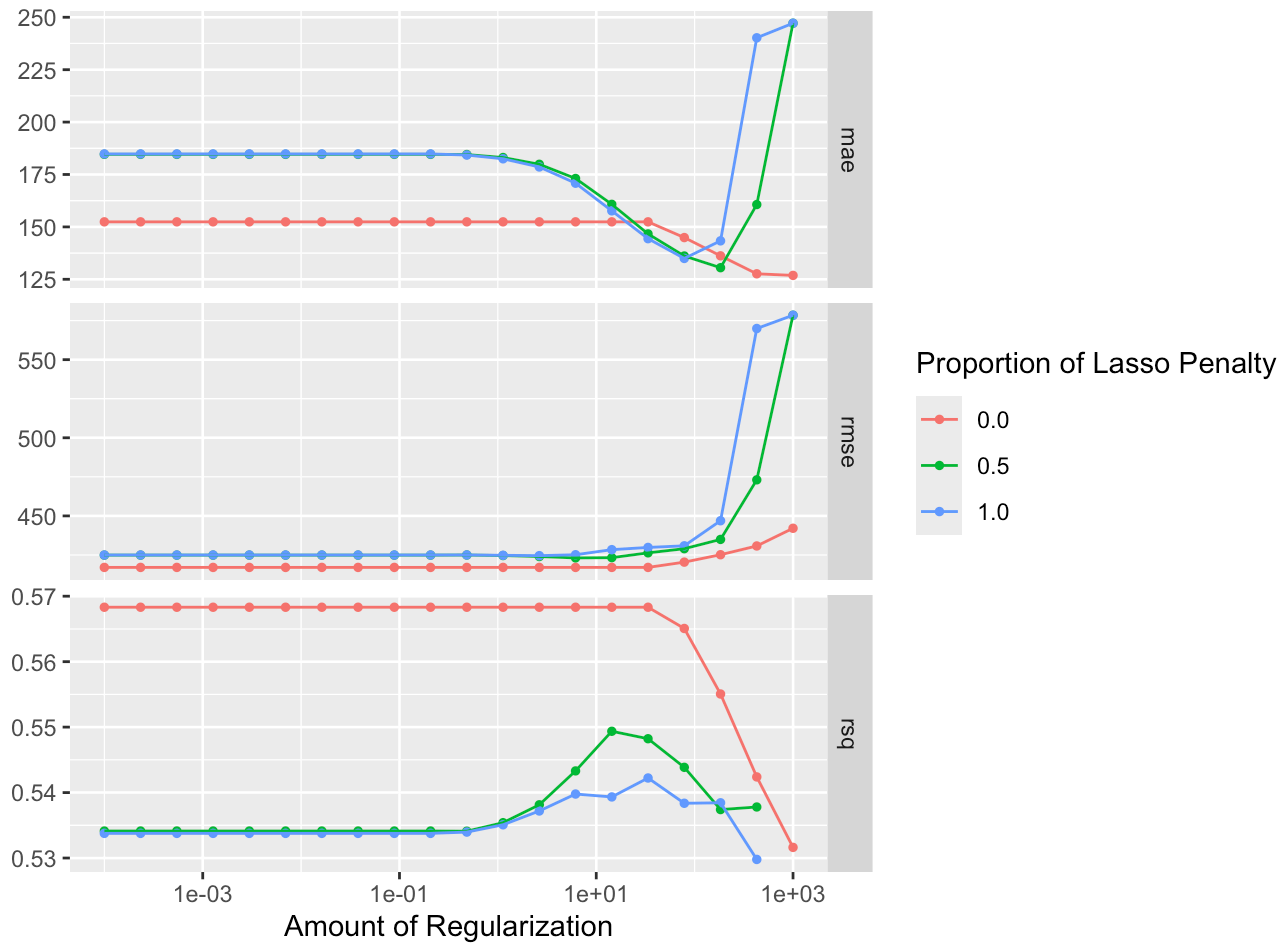
```
reg_cv_d = tune_grid(  
  reg_wf_d,  
  resamples = folds_grouped,  
  grid = reg_grid,  
  metrics = reg_metrics,  
  control = control_grid())
```

→ A | warning: A correlation computation is required, but `estimate` is constant and has 0 standard deviation, resulting in a divide by 0 error. `NA` will be returned.

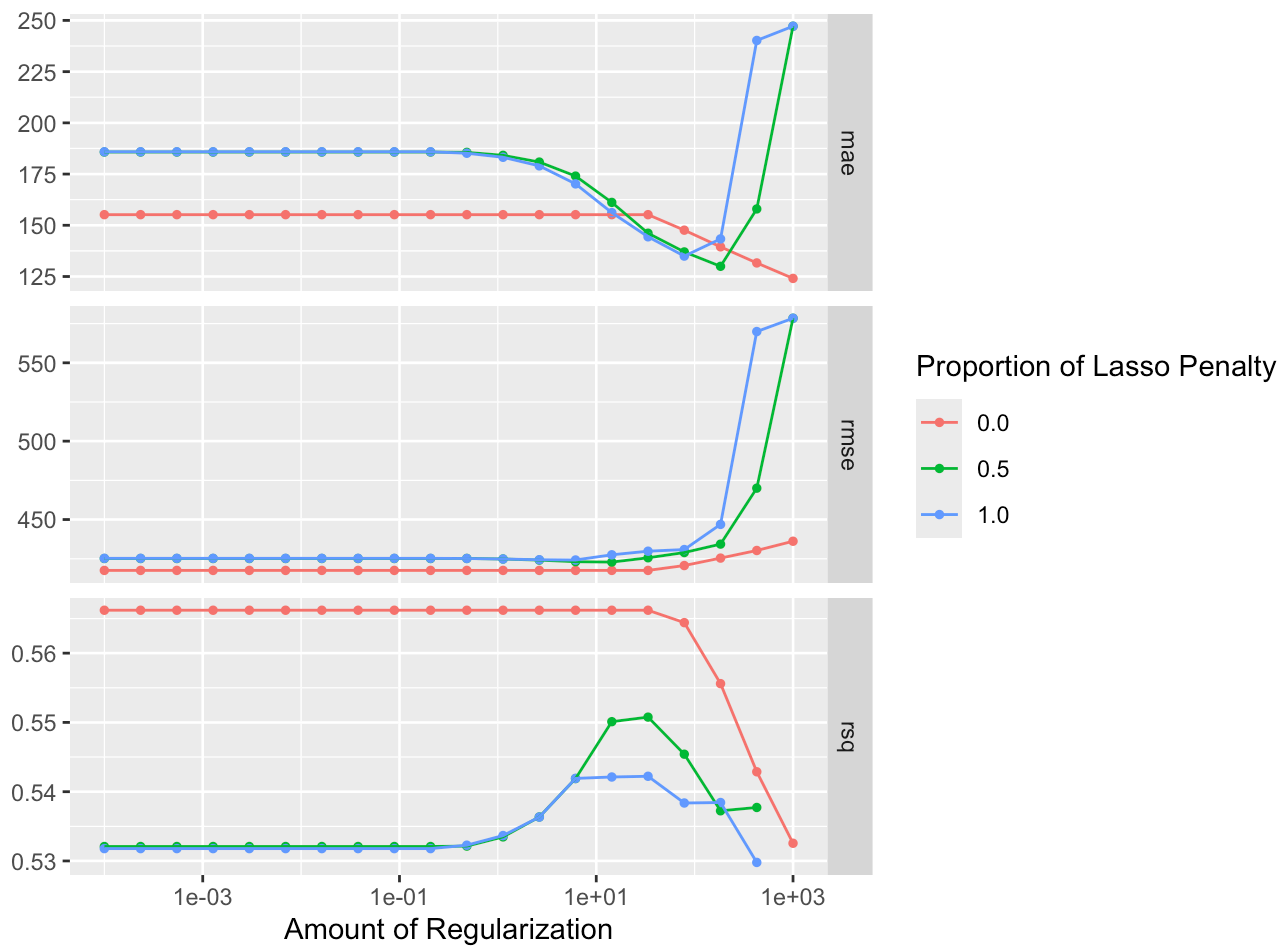
There were issues with some computations A: x15

There were issues with some computations A: x30  
There were issues with some computations A: x30

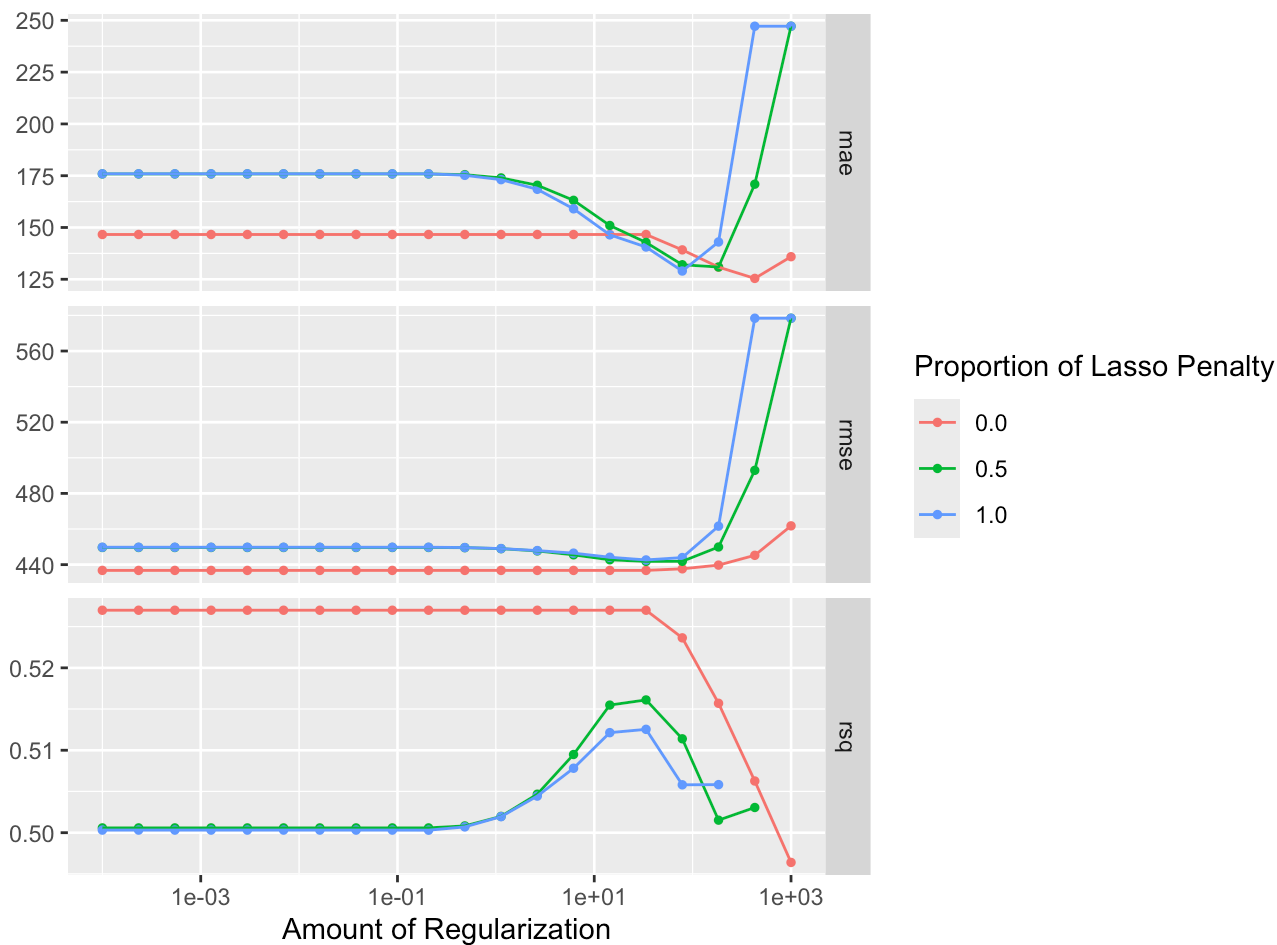
```
autoplot(reg_cv_a)
```



```
autoplot(reg_cv_c)
```



```
autoplot(reg_cv_d)
```



```
# best metrics, MAE and RMSE
```

```
show_best(reg_cv_a, metric = "mae", n = 5)
```

```
# A tibble: 5 × 8
```

|   | penalty | mixture | .metric | .estimator | mean  | n     | std_err | .config          |
|---|---------|---------|---------|------------|-------|-------|---------|------------------|
|   | <dbl>   | <dbl>   | <chr>   | <chr>      | <dbl> | <int> | <dbl>   | <chr>            |
| 1 | 1000    | 0       | mae     | standard   | 127.  | 10    | 5.50    | pre0_mod58_post0 |
| 2 | 428.    | 0       | mae     | standard   | 128.  | 10    | 4.27    | pre0_mod55_post0 |
| 3 | 183.    | 0.5     | mae     | standard   | 131.  | 10    | 5.15    | pre0_mod53_post0 |
| 4 | 78.5    | 1       | mae     | standard   | 135.  | 10    | 4.26    | pre0_mod51_post0 |
| 5 | 78.5    | 0.5     | mae     | standard   | 136.  | 10    | 3.56    | pre0_mod50_post0 |

```
show_best(reg_cv_c, metric = "mae", n = 5)
```

```
# A tibble: 5 × 8
```

|   | penalty | mixture | .metric | .estimator | mean  | n     | std_err | .config          |
|---|---------|---------|---------|------------|-------|-------|---------|------------------|
|   | <dbl>   | <dbl>   | <chr>   | <chr>      | <dbl> | <int> | <dbl>   | <chr>            |
| 1 | 1000    | 0       | mae     | standard   | 124.  | 10    | 4.66    | pre0_mod58_post0 |
| 2 | 183.    | 0.5     | mae     | standard   | 130.  | 10    | 4.95    | pre0_mod53_post0 |
| 3 | 428.    | 0       | mae     | standard   | 132.  | 10    | 3.78    | pre0_mod55_post0 |
| 4 | 78.5    | 1       | mae     | standard   | 135.  | 10    | 4.26    | pre0_mod51_post0 |
| 5 | 78.5    | 0.5     | mae     | standard   | 137.  | 10    | 3.45    | pre0_mod50_post0 |

```
show_best(reg_cv_d, metric = "mae", n = 5)
```

```
# A tibble: 5 × 8
```

|   | penalty | mixture | .metric | .estimator | mean  | n     | std_err | .config          |
|---|---------|---------|---------|------------|-------|-------|---------|------------------|
|   | <dbl>   | <dbl>   | <chr>   | <chr>      | <dbl> | <int> | <dbl>   | <chr>            |
| 1 | 428.    | 0       | mae     | standard   | 125.  | 10    | 4.82    | pre0_mod55_post0 |
| 2 | 78.5    | 1       | mae     | standard   | 129.  | 10    | 4.31    | pre0_mod51_post0 |
| 3 | 183.    | 0       | mae     | standard   | 131.  | 10    | 4.05    | pre0_mod52_post0 |
| 4 | 183.    | 0.5     | mae     | standard   | 131.  | 10    | 5.43    | pre0_mod53_post0 |
| 5 | 78.5    | 0.5     | mae     | standard   | 132.  | 10    | 3.55    | pre0_mod50_post0 |

```
show_best(reg_cv_a, metric = "rmse", n = 5)
```

```
# A tibble: 5 × 8
```

|   | penalty  | mixture | .metric | .estimator | mean  | n     | std_err | .config          |
|---|----------|---------|---------|------------|-------|-------|---------|------------------|
|   | <dbl>    | <dbl>   | <chr>   | <chr>      | <dbl> | <int> | <dbl>   | <chr>            |
| 1 | 0.0001   | 0       | rmse    | standard   | 417.  | 10    | 63.5    | pre0_mod01_post0 |
| 2 | 0.000234 | 0       | rmse    | standard   | 417.  | 10    | 63.5    | pre0_mod04_post0 |
| 3 | 0.000546 | 0       | rmse    | standard   | 417.  | 10    | 63.5    | pre0_mod07_post0 |
| 4 | 0.00127  | 0       | rmse    | standard   | 417.  | 10    | 63.5    | pre0_mod10_post0 |
| 5 | 0.00298  | 0       | rmse    | standard   | 417.  | 10    | 63.5    | pre0_mod13_post0 |

```
show_best(reg_cv_c, metric = "rmse", n = 5)
```

```
# A tibble: 5 × 8
```

|   | penalty  | mixture | .metric | .estimator | mean  | n     | std_err | .config          |
|---|----------|---------|---------|------------|-------|-------|---------|------------------|
|   | <dbl>    | <dbl>   | <chr>   | <chr>      | <dbl> | <int> | <dbl>   | <chr>            |
| 1 | 0.0001   | 0       | rmse    | standard   | 418.  | 10    | 61.7    | pre0_mod01_post0 |
| 2 | 0.000234 | 0       | rmse    | standard   | 418.  | 10    | 61.7    | pre0_mod04_post0 |
| 3 | 0.000546 | 0       | rmse    | standard   | 418.  | 10    | 61.7    | pre0_mod07_post0 |
| 4 | 0.00127  | 0       | rmse    | standard   | 418.  | 10    | 61.7    | pre0_mod10_post0 |
| 5 | 0.00298  | 0       | rmse    | standard   | 418.  | 10    | 61.7    | pre0_mod13_post0 |

```
show_best(reg_cv_d, metric = "rmse", n = 5)
```

```
# A tibble: 5 × 8
```

|   | penalty  | mixture | .metric | .estimator | mean  | n     | std_err | .config          |
|---|----------|---------|---------|------------|-------|-------|---------|------------------|
|   | <dbl>    | <dbl>   | <chr>   | <chr>      | <dbl> | <int> | <dbl>   | <chr>            |
| 1 | 0.0001   | 0       | rmse    | standard   | 437.  | 10    | 75.6    | pre0_mod01_post0 |
| 2 | 0.000234 | 0       | rmse    | standard   | 437.  | 10    | 75.6    | pre0_mod04_post0 |
| 3 | 0.000546 | 0       | rmse    | standard   | 437.  | 10    | 75.6    | pre0_mod07_post0 |
| 4 | 0.00127  | 0       | rmse    | standard   | 437.  | 10    | 75.6    | pre0_mod10_post0 |
| 5 | 0.00298  | 0       | rmse    | standard   | 437.  | 10    | 75.6    | pre0_mod13_post0 |

```
# Finalize the MAE-selected configurations
```

```
best_a = select_best(reg_cv_a, metric = "mae")
best_c = select_best(reg_cv_c, metric = "mae")
best_d = select_best(reg_cv_d, metric = "mae")
```

```

reg_fit_a = reg_wf_a |>
  finalize_workflow(best_a) |>
  fit(data = train_data)
reg_fit_c = reg_wf_c |>
  finalize_workflow(best_c) |>
  fit(data = train_data)
reg_fit_d = reg_wf_d |>
  finalize_workflow(best_d) |>
  fit(data = train_data)

```

```
# Examine variation in predictions
```

```

bind_rows(
  predict(reg_fit_a, new_data = train_data) |>
    mutate(model = "A"),
  predict(reg_fit_c, new_data = train_data) |>
    mutate(model = "C"),
  predict(reg_fit_d, new_data = train_data) |>
    mutate(model = "D")) |>
  group_by(model) |>
  summarize(
    min_prediction = min(.pred),
    median_prediction = median(.pred),
    max_prediction = max(.pred),
    prediction_sd = sd(.pred),
    .groups = "drop"
  )

```

```
# A tibble: 3 × 5
```

|   | model | min_prediction | median_prediction | max_prediction | prediction_sd |
|---|-------|----------------|-------------------|----------------|---------------|
|   | <chr> | <dbl>          | <dbl>             | <dbl>          | <dbl>         |
| 1 | A     | -46.2          | 139.              | 4457.          | 266.          |
| 2 | C     | -43.2          | 120.              | 5367.          | 305.          |
| 3 | D     | -81.2          | 128.              | 3816.          | 287.          |

```
# Training predictions for both models
```

```

reg_predictions = bind_rows(
  train_data |>
    select(ZIP, year, private_k12) |>
    bind_cols(predict(reg_fit_a, new_data = train_data)) |>
    mutate(model = "A"),
  train_data |>
    select(ZIP, year, private_k12) |>
    bind_cols(predict(reg_fit_c, new_data = train_data)) |>
    mutate(model = "C"),
  train_data |>
    select(ZIP, year, private_k12) |>
    bind_cols(predict(reg_fit_d, new_data = train_data)) |>
    mutate(model = "D"), )

```

```
# Count impossible predictions
```

```
reg_predictions |>
  group_by(model) |>
  summarize(
    negative_predictions = sum(.pred < 0),
    pct_negative = mean(.pred < 0) * 100,
    .groups = "drop")
```

# A tibble: 3 × 3

|   | model | negative_predictions | pct_negative |
|---|-------|----------------------|--------------|
|   | <chr> | <int>                | <dbl>        |
| 1 | A     | 404                  | 1.34         |
| 2 | C     | 1587                 | 5.28         |
| 3 | D     | 1925                 | 6.40         |

```
reg_metrics_a = collect_metrics(reg_cv_a) |>
  semi_join(best_a, by = c("penalty", "mixture")) |>
  mutate(model = "Regularized A")
reg_metrics_c = collect_metrics(reg_cv_c) |>
  semi_join(best_c, by = c("penalty", "mixture")) |>
  mutate(model = "Regularized C")
reg_metrics_d = collect_metrics(reg_cv_d) |>
  semi_join(best_d, by = c("penalty", "mixture")) |>
  mutate(model = "Regularized D")
```

```
ols_vs_regularized = bind_rows(
  collect_metrics(lr_cv_a) |>
    mutate(model = "OLS A"),
  collect_metrics(lr_cv_c) |>
    mutate(model = "OLS C"),
  collect_metrics(lr_cv_d) |>
    mutate(model = "OLS D"),
  reg_metrics_a,
  reg_metrics_c,
  reg_metrics_d) |>
  dplyr::select(model, .metric, mean, std_err) |>
  arrange(.metric, mean)
```

```
ols_vs_regularized
```

# A tibble: 18 × 4

|   | model         | .metric | mean  | std_err |
|---|---------------|---------|-------|---------|
|   | <chr>         | <chr>   | <dbl> | <dbl>   |
| 1 | Regularized C | mae     | 124.  | 4.66    |
| 2 | Regularized D | mae     | 125.  | 4.82    |
| 3 | Regularized A | mae     | 127.  | 5.50    |
| 4 | OLS D         | mae     | 177.  | 3.09    |
| 5 | OLS A         | mae     | 186.  | 3.31    |
| 6 | OLS C         | mae     | 187.  | 3.19    |
| 7 | OLS A         | rmse    | 425.  | 47.2    |
| 8 | OLS C         | rmse    | 425.  | 46.4    |
| 9 | Regularized C | rmse    | 436.  | 83.8    |

|    |               |      |       |        |
|----|---------------|------|-------|--------|
| 10 | Regularized A | rmse | 442.  | 87.2   |
| 11 | Regularized D | rmse | 445.  | 89.0   |
| 12 | OLS D         | rmse | 450.  | 62.6   |
| 13 | OLS D         | rsq  | 0.500 | 0.0256 |
| 14 | Regularized D | rsq  | 0.506 | 0.0452 |
| 15 | OLS C         | rsq  | 0.531 | 0.0384 |
| 16 | Regularized A | rsq  | 0.532 | 0.0365 |
| 17 | Regularized C | rsq  | 0.533 | 0.0345 |
| 18 | OLS A         | rsq  | 0.533 | 0.0371 |

```
best_a
```

```
# A tibble: 1 × 3
  penalty mixture .config
  <dbl>   <dbl> <chr>
1    1000     0 pre0_mod58_post0
```

```
best_c
```

```
# A tibble: 1 × 3
  penalty mixture .config
  <dbl>   <dbl> <chr>
1    1000     0 pre0_mod58_post0
```

```
best_d
```

```
# A tibble: 1 × 3
  penalty mixture .config
  <dbl>   <dbl> <chr>
1    428.     0 pre0_mod55_post0
```

```
tidy(reg_fit_a) |>
  filter(term != "(Intercept)") |>
  arrange(desc(abs(estimate)))
```

Attaching package: 'Matrix'

The following objects are masked from 'package:tidyr':

expand, pack, unpack

Loaded glmnet 5.0

```
# A tibble: 22 × 3
  term                                estimate penalty
  <chr>                                <dbl>   <dbl>
1 age_0_4                             112.    1000
2 school_age_population                90.8    1000
3 total_households                     50.7    1000
```

```

4 median_home_value_inflation_adjusted 28.6 1000
5 pct_adults_grad_plus 11.5 1000
6 pct_households_with_children 11.2 1000
7 pct_adults_bachelors_plus 10.6 1000
8 state_PA 7.56 1000
9 state_MA -7.53 1000
10 state_NY 7.06 1000
# i 12 more rows

```

```

tidy(reg_fit_c) |>
  filter(term != "(Intercept)") |>
  arrange(desc(abs(estimate)))

```

# A tibble: 23 × 3

|    | term<br><chr>                        | estimate<br><dbl> | penalty<br><dbl> |
|----|--------------------------------------|-------------------|------------------|
| 1  | age_0_4                              | 87.4              | 1000             |
| 2  | age_5_9                              | 74.7              | 1000             |
| 3  | age_10_14                            | 60.7              | 1000             |
| 4  | age_15_17                            | 48.1              | 1000             |
| 5  | total_households                     | 28.7              | 1000             |
| 6  | median_home_value_inflation_adjusted | 26.9              | 1000             |
| 7  | pct_adults_grad_plus                 | 12.6              | 1000             |
| 8  | state_PA                             | 7.84              | 1000             |
| 9  | state_MA                             | -7.44             | 1000             |
| 10 | state_NY                             | 6.29              | 1000             |

# i 13 more rows

```

tidy(reg_fit_d) |>
  filter(term != "(Intercept)") |>
  arrange(desc(abs(estimate)))

```

# A tibble: 20 × 3

|    | term<br><chr>                        | estimate<br><dbl> | penalty<br><dbl> |
|----|--------------------------------------|-------------------|------------------|
| 1  | school_age_population                | 181.              | 428.             |
| 2  | total_households                     | 87.2              | 428.             |
| 3  | median_home_value_inflation_adjusted | 49.4              | 428.             |
| 4  | pct_households_with_children         | 16.7              | 428.             |
| 5  | pct_adults_grad_plus                 | 16.5              | 428.             |
| 6  | state_PA                             | 13.6              | 428.             |
| 7  | state_MA                             | -12.9             | 428.             |
| 8  | median_hh_income_inflation_adjusted  | -11.1             | 428.             |
| 9  | state_NY                             | 10.3              | 428.             |
| 10 | pct_adults_hs_plus                   | -8.70             | 428.             |
| 11 | state_NJ                             | -6.26             | 428.             |
| 12 | state_MD                             | 5.81              | 428.             |
| 13 | state_VA                             | -5.26             | 428.             |
| 14 | state_DE                             | 1.80              | 428.             |
| 15 | state_NH                             | -1.57             | 428.             |
| 16 | state_WV                             | -1.45             | 428.             |

|             |        |      |
|-------------|--------|------|
| 17 state_VT | -1.39  | 428. |
| 18 state_ME | 0.453  | 428. |
| 19 state_RI | -0.364 | 428. |
| 20 state_DC | 0.353  | 428. |

I like, based on MAE and RSQ, Regularized model C most. Interestingly, RMSE and MAE give different best models.

## Ensemble Methods

Let's move on and do our ensemble methods with trees, random forests, and boosting.

```
tune_xgboost_spec = boost_tree(
  mode = "regression",
  trees = tune(),
  learn_rate = tune(),
  tree_depth = tune()) |>
  set_engine("xgboost")

tune_xgboost_wf_a = workflow() |>
  add_recipe(recipe_a) |>
  add_model(tune_xgboost_spec)
tune_xgboost_wf_c = workflow() |>
  add_recipe(recipe_c) |>
  add_model(tune_xgboost_spec)
tune_xgboost_wf_d = workflow() |>
  add_recipe(recipe_d) |>
  add_model(tune_xgboost_spec)

xgboost_params_a = extract_parameter_set_dials(tune_xgboost_wf_a) |>
  update(trees = trees(c(50, 500)),
         learn_rate = learn_rate(c(-3, -1)),
         tree_depth = tree_depth(c(1, 5)))
xgboost_params_c = extract_parameter_set_dials(tune_xgboost_wf_c) |>
  update(trees = trees(c(50, 500)),
         learn_rate = learn_rate(c(-3, -1)),
         tree_depth = tree_depth(c(1, 5)))
xgboost_params_d = extract_parameter_set_dials(tune_xgboost_wf_d) |>
  update(trees = trees(c(50, 500)),
         learn_rate = learn_rate(c(-3, -1)),
         tree_depth = tree_depth(c(1, 5)))
```

```
tune_results_xgboost_a = tune_grid(
  tune_xgboost_wf_a,
  resamples = folds_grouped,
  metrics = reg_metrics,
  grid = grid_random(xgboost_params_a, size = 20))
tune_results_xgboost_c = tune_grid(
  tune_xgboost_wf_c,
  resamples = folds_grouped,
  metrics = reg_metrics,
```

```

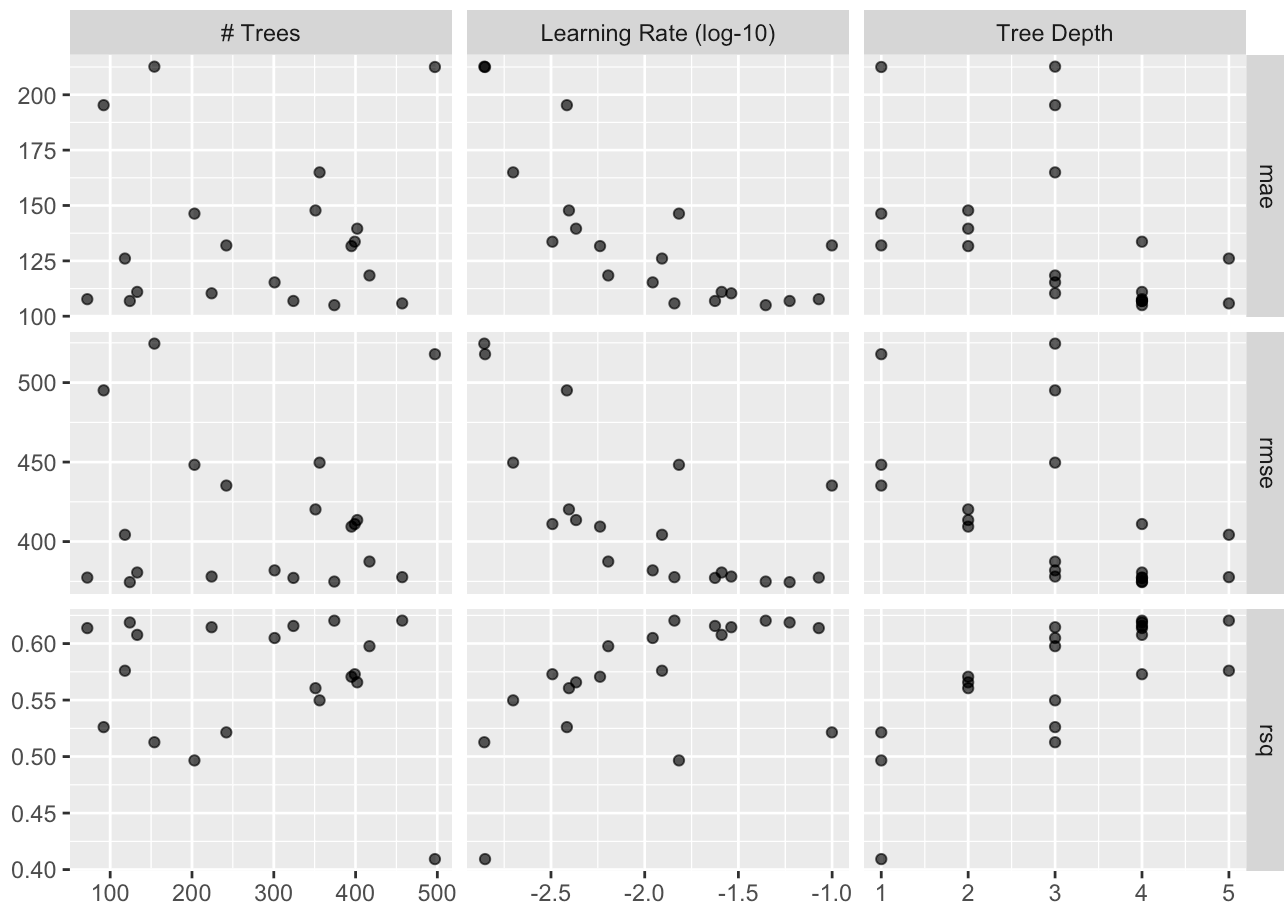
grid = grid_random(xgboost_params_a, size = 20))
tune_results_xgboost_d = tune_grid(
  tune_xgboost_wf_d,
  resamples = folds_grouped,
  metrics = reg_metrics,
  grid = grid_random(xgboost_params_a, size = 20))

```

```

autoplot(tune_results_xgboost_a)

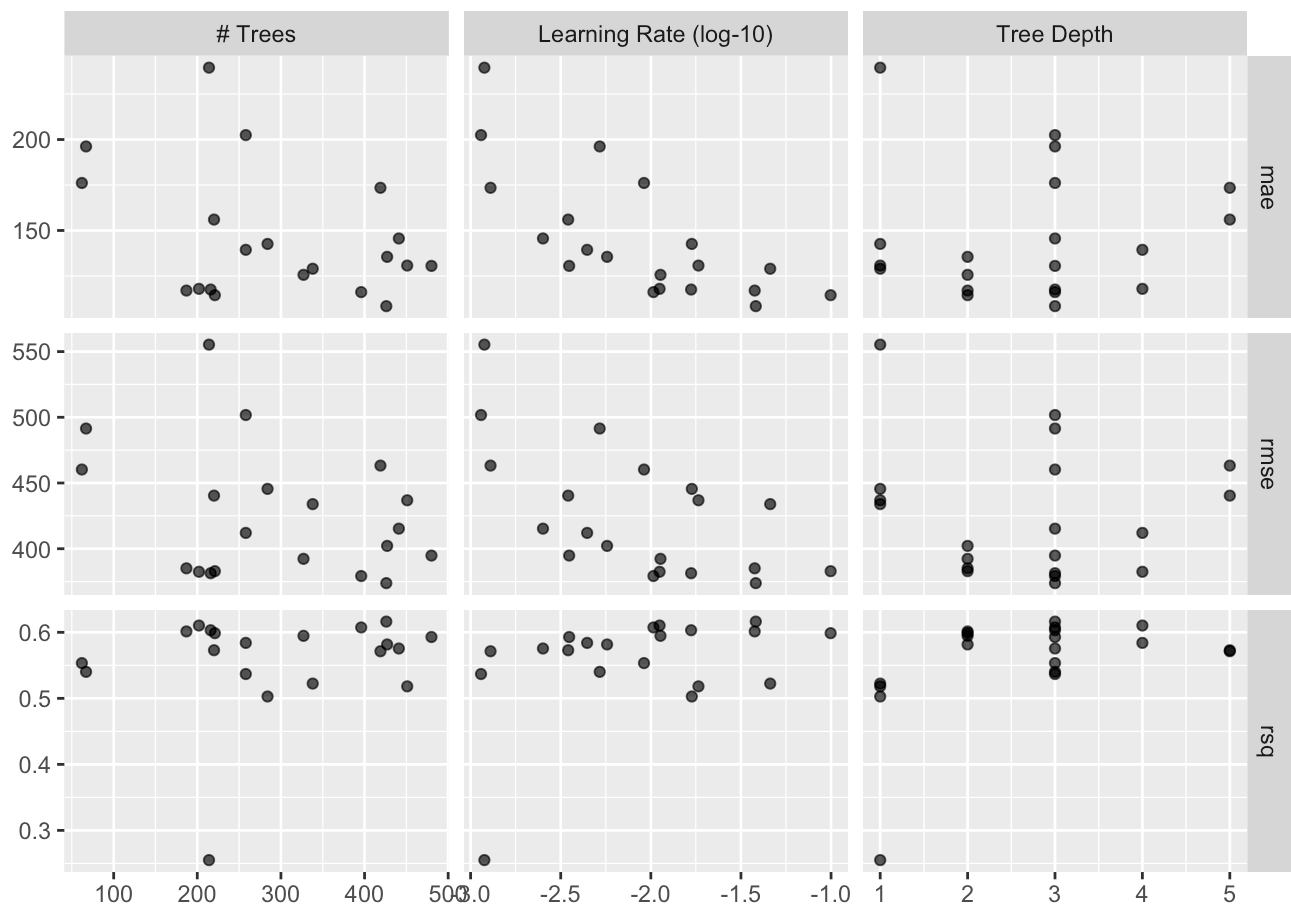
```



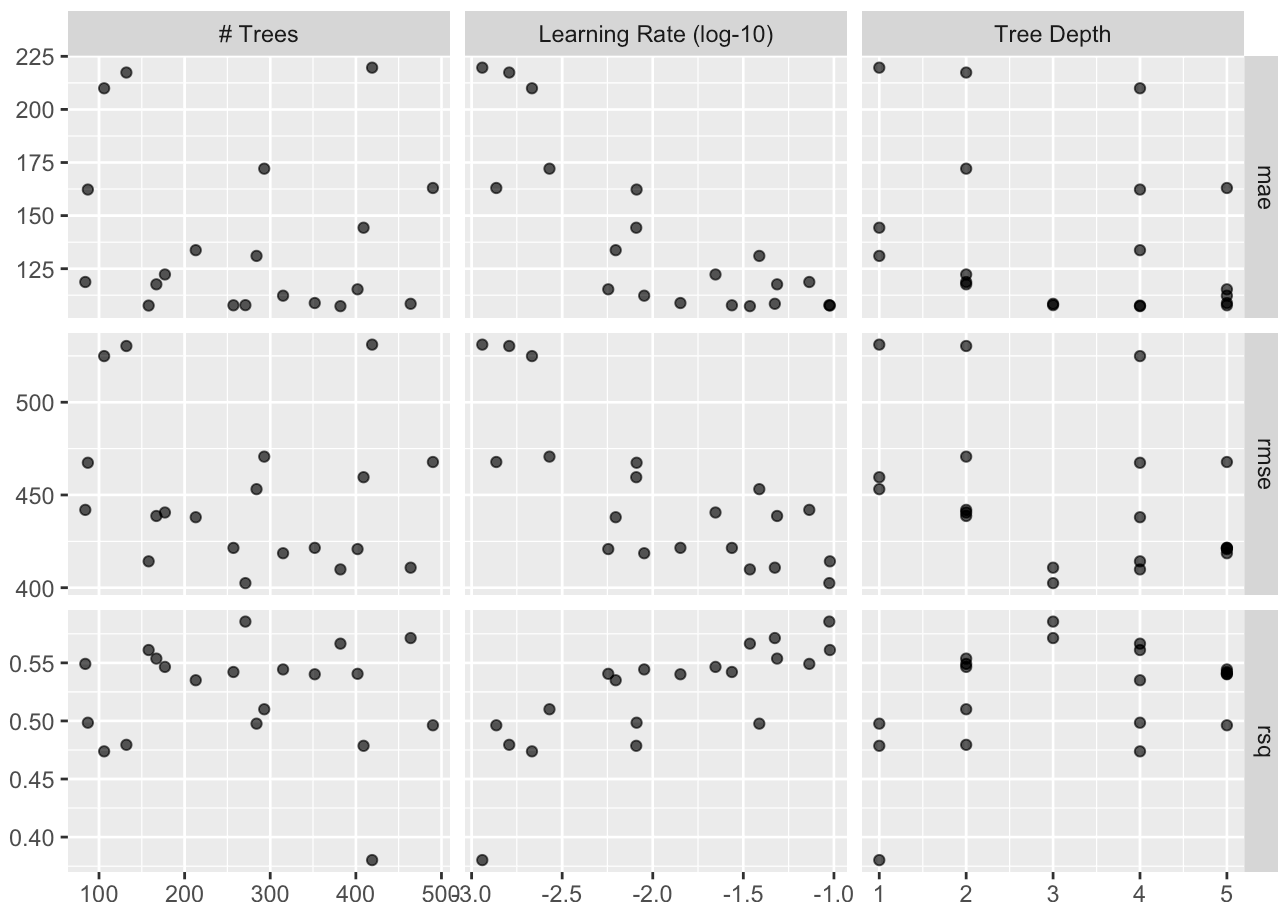
```

autoplot(tune_results_xgboost_c)

```



```
autoplot(tune_results_xgboost_d)
```



```
show_best(tune_results_xgboost_a, metric = "mae", n = 5)
```

# A tibble: 5 × 9

|   | trees | tree_depth | learn_rate | .metric | .estimator | mean  | n     | std_err | .config       |
|---|-------|------------|------------|---------|------------|-------|-------|---------|---------------|
|   | <int> | <int>      | <dbl>      | <chr>   | <chr>      | <dbl> | <int> | <dbl>   | <chr>         |
| 1 | 374   | 4          | 0.0442     | mae     | standard   | 105.  | 10    | 4.29    | pre0_mod14... |
| 2 | 457   | 5          | 0.0144     | mae     | standard   | 106.  | 10    | 4.10    | pre0_mod19... |
| 3 | 124   | 4          | 0.0593     | mae     | standard   | 107.  | 10    | 4.12    | pre0_mod04... |
| 4 | 324   | 4          | 0.0237     | mae     | standard   | 107.  | 10    | 4.19    | pre0_mod11... |
| 5 | 72    | 4          | 0.0849     | mae     | standard   | 108.  | 10    | 4.15    | pre0_mod01... |

```
show_best(tune_results_xgboost_a, metric = "rmse", n = 5)
```

# A tibble: 5 × 9

|   | trees | tree_depth | learn_rate | .metric | .estimator | mean  | n     | std_err | .config       |
|---|-------|------------|------------|---------|------------|-------|-------|---------|---------------|
|   | <int> | <int>      | <dbl>      | <chr>   | <chr>      | <dbl> | <int> | <dbl>   | <chr>         |
| 1 | 124   | 4          | 0.0593     | rmse    | standard   | 374.  | 10    | 67.5    | pre0_mod04... |
| 2 | 374   | 4          | 0.0442     | rmse    | standard   | 375.  | 10    | 68.0    | pre0_mod14... |
| 3 | 324   | 4          | 0.0237     | rmse    | standard   | 377.  | 10    | 68.6    | pre0_mod11... |
| 4 | 72    | 4          | 0.0849     | rmse    | standard   | 377.  | 10    | 69.4    | pre0_mod01... |
| 5 | 457   | 5          | 0.0144     | rmse    | standard   | 378.  | 10    | 68.5    | pre0_mod19... |

```
show_best(tune_results_xgboost_a, metric = "rsq", n = 5)
```

```
# A tibble: 5 × 9
  trees tree_depth learn_rate .metric .estimator mean      n std_err .config
  <int>    <int>      <dbl> <chr>   <chr>    <dbl> <int>  <dbl> <chr>
1   457         5    0.0144 rsq     standard 0.620   10  0.0475 pre0_mod19...
2   374         4    0.0442 rsq     standard 0.620   10  0.0496 pre0_mod14...
3   124         4    0.0593 rsq     standard 0.619   10  0.0485 pre0_mod04...
4   324         4    0.0237 rsq     standard 0.616   10  0.0498 pre0_mod11...
5   224         3    0.0290 rsq     standard 0.614   10  0.0498 pre0_mod08...
```

```
show_best(tune_results_xgboost_c, metric = "mae", n = 5)
```

```
# A tibble: 5 × 9
  trees tree_depth learn_rate .metric .estimator mean      n std_err .config
  <int>    <int>      <dbl> <chr>   <chr>    <dbl> <int>  <dbl> <chr>
1   426         3    0.0382 mae     standard 108.    10   3.72 pre0_mod16...
2   221         2    0.0995 mae     standard 114.    10   3.52 pre0_mod08...
3   396         3    0.0103 mae     standard 116.    10   3.63 pre0_mod14...
4   187         2    0.0377 mae     standard 117.    10   3.49 pre0_mod03...
5   216         3    0.0167 mae     standard 118.    10   3.67 pre0_mod06...
```

```
show_best(tune_results_xgboost_c, metric = "rmse", n = 5)
```

```
# A tibble: 5 × 9
  trees tree_depth learn_rate .metric .estimator mean      n std_err .config
  <int>    <int>      <dbl> <chr>   <chr>    <dbl> <int>  <dbl> <chr>
1   426         3    0.0382 rmse     standard 374.    10  62.1 pre0_mod16...
2   396         3    0.0103 rmse     standard 379.    10  61.4 pre0_mod14...
3   216         3    0.0167 rmse     standard 381.    10  61.7 pre0_mod06...
4   202         4    0.0112 rmse     standard 382.    10  63.4 pre0_mod04...
5   221         2    0.0995 rmse     standard 383.    10  65.7 pre0_mod08...
```

```
show_best(tune_results_xgboost_c, metric = "rsq", n = 5)
```

```
# A tibble: 5 × 9
  trees tree_depth learn_rate .metric .estimator mean      n std_err .config
  <int>    <int>      <dbl> <chr>   <chr>    <dbl> <int>  <dbl> <chr>
1   426         3    0.0382 rsq     standard 0.616   10  0.0506 pre0_mod16...
2   202         4    0.0112 rsq     standard 0.610   10  0.0543 pre0_mod04...
3   396         3    0.0103 rsq     standard 0.607   10  0.0493 pre0_mod14...
4   216         3    0.0167 rsq     standard 0.603   10  0.0499 pre0_mod06...
5   187         2    0.0377 rsq     standard 0.601   10  0.0443 pre0_mod03...
```

```
show_best(tune_results_xgboost_d, metric = "mae", n = 5)
```

```
# A tibble: 5 × 9
  trees tree_depth learn_rate .metric .estimator mean      n std_err .config
  <int>    <int>      <dbl> <chr>   <chr>    <dbl> <int>  <dbl> <chr>
1   382         4    0.0344 mae     standard 107.    10   4.39 pre0_mod15...
2   158         4    0.0950 mae     standard 108.    10   4.42 pre0_mod05...
3   257         5    0.0273 mae     standard 108.    10   4.44 pre0_mod09...
```

|   |     |   |        |     |          |      |    |      |               |
|---|-----|---|--------|-----|----------|------|----|------|---------------|
| 4 | 271 | 3 | 0.0943 | mae | standard | 108. | 10 | 3.91 | pre0_mod10... |
| 5 | 464 | 3 | 0.0472 | mae | standard | 108. | 10 | 4.03 | pre0_mod19... |

```
show_best(tune_results_xgboost_d, metric = "rmse", n = 5)
```

```
# A tibble: 5 × 9
  trees tree_depth learn_rate .metric .estimator mean      n std_err .config
  <int>    <int>      <dbl> <chr>  <chr>    <dbl> <int>  <dbl> <chr>
1   271        3    0.0943 rmse   standard  402.    10    75.4 pre0_mod10...
2   382        4    0.0344 rmse   standard  410.    10    80.6 pre0_mod15...
3   464        3    0.0472 rmse   standard  411.    10    75.8 pre0_mod19...
4   158        4    0.0950 rmse   standard  414.    10    78.7 pre0_mod05...
5   315        5    0.00897 rmse   standard  419.    10    85.0 pre0_mod13...
```

```
show_best(tune_results_xgboost_d, metric = "rsq", n = 5)
```

```
# A tibble: 5 × 9
  trees tree_depth learn_rate .metric .estimator mean      n std_err .config
  <int>    <int>      <dbl> <chr>  <chr>    <dbl> <int>  <dbl> <chr>
1   271        3    0.0943 rsq     standard  0.586    10  0.0571 pre0_mod10...
2   464        3    0.0472 rsq     standard  0.571    10  0.0607 pre0_mod19...
3   382        4    0.0344 rsq     standard  0.567    10  0.0622 pre0_mod15...
4   158        4    0.0950 rsq     standard  0.561    10  0.0626 pre0_mod05...
5   167        2    0.0486 rsq     standard  0.554    10  0.0668 pre0_mod06...
```

```
best_xgboost_a = select_best(tune_results_xgboost_a, metric = "mae")
best_xgboost_c = select_best(tune_results_xgboost_c, metric = "mae")
best_xgboost_d = select_best(tune_results_xgboost_d, metric = "mae")
```

```
best_xgboost_a
```

```
# A tibble: 1 × 4
  trees tree_depth learn_rate .config
  <int>    <int>      <dbl> <chr>
1   374        4    0.0442 pre0_mod14_post0
```

```
best_xgboost_c
```

```
# A tibble: 1 × 4
  trees tree_depth learn_rate .config
  <int>    <int>      <dbl> <chr>
1   426        3    0.0382 pre0_mod16_post0
```

```
best_xgboost_d
```

```
# A tibble: 1 × 4
  trees tree_depth learn_rate .config
  <int>    <int>      <dbl> <chr>
1   382        4    0.0344 pre0_mod15_post0
```

We can now move onto random forests

```
tune_rf_spec = rand_forest(
  mode = "regression",
  mtry = tune(),
  min_n = tune(),
  trees = 300) |>
  set_engine("ranger", importance = "impurity", num.threads = 1)

tune_rf_wf_a = workflow() |>
  add_recipe(recipe_a) |>
  add_model(tune_rf_spec)
tune_rf_wf_c = workflow() |>
  add_recipe(recipe_c) |>
  add_model(tune_rf_spec)
tune_rf_wf_d = workflow() |>
  add_recipe(recipe_d) |>
  add_model(tune_rf_spec)

processed_train_a = prep(recipe_a) |>
  bake(new_data = NULL) |>
  select(-private_k12)
processed_train_c = prep(recipe_c) |>
  bake(new_data = NULL) |>
  select(-private_k12)
processed_train_d = prep(recipe_d) |>
  bake(new_data = NULL) |>
  select(-private_k12)

rf_params_a = extract_parameter_set_dials(tune_rf_wf_a) |>
  finalize(processed_train_a) |>
  update(min_n = min_n(c(2, 25)))
rf_params_c = extract_parameter_set_dials(tune_rf_wf_c) |>
  finalize(processed_train_c) |>
  update(min_n = min_n(c(2, 25)))
rf_params_d = extract_parameter_set_dials(tune_rf_wf_d) |>
  finalize(processed_train_d) |>
  update(min_n = min_n(c(2, 25)))
```

```
set.seed(88)
```

```
tune_results_rf_a = tune_grid(
  tune_rf_wf_a,
  resamples = folds_grouped,
  metrics = reg_metrics,
  grid = grid_random(rf_params_a, size = 20))
```

```
set.seed(88)
```

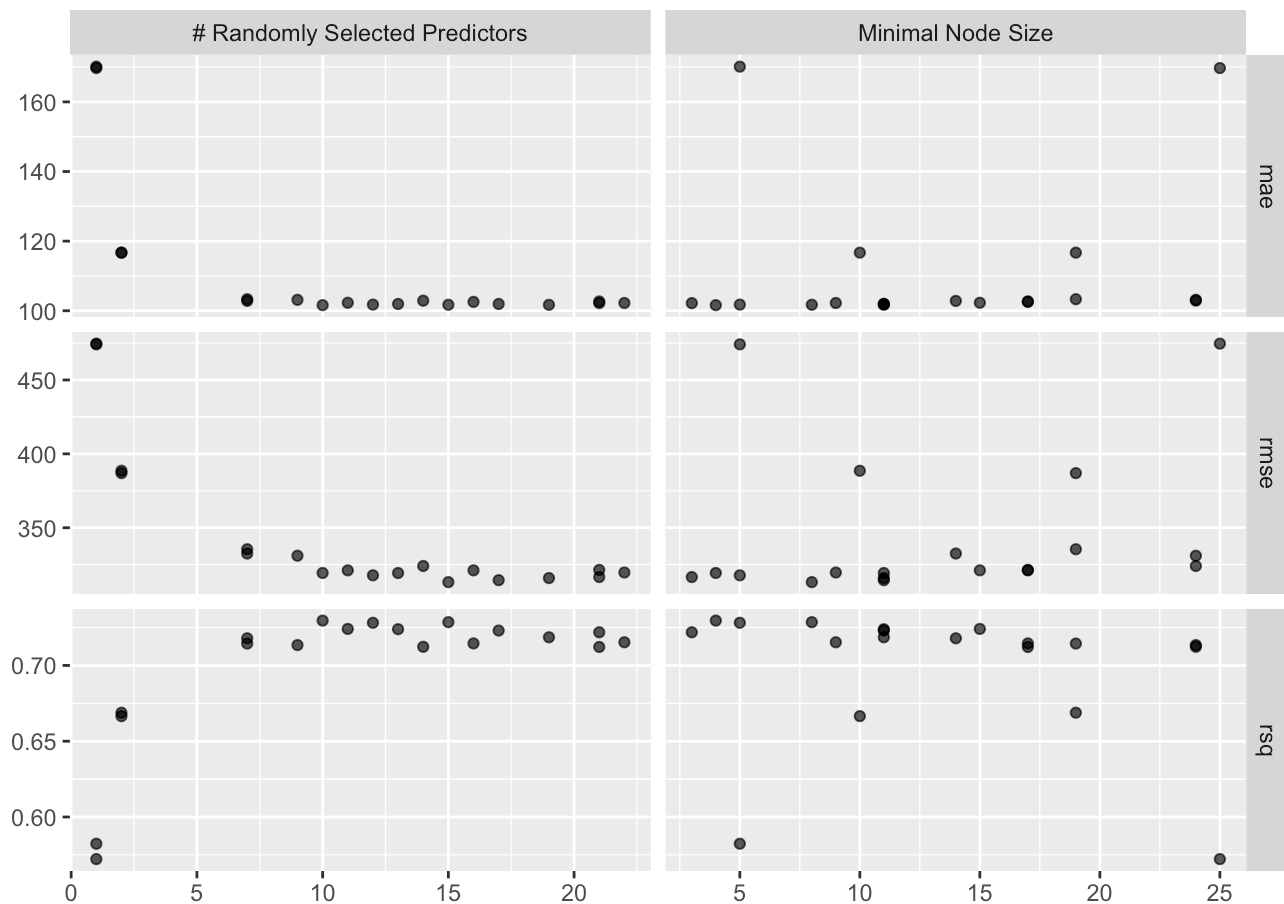
```
tune_results_rf_c = tune_grid(
  tune_rf_wf_c,
```

```
resamples = folds_grouped,
metrics = reg_metrics,
grid = grid_random(rf_params_c, size = 20))
```

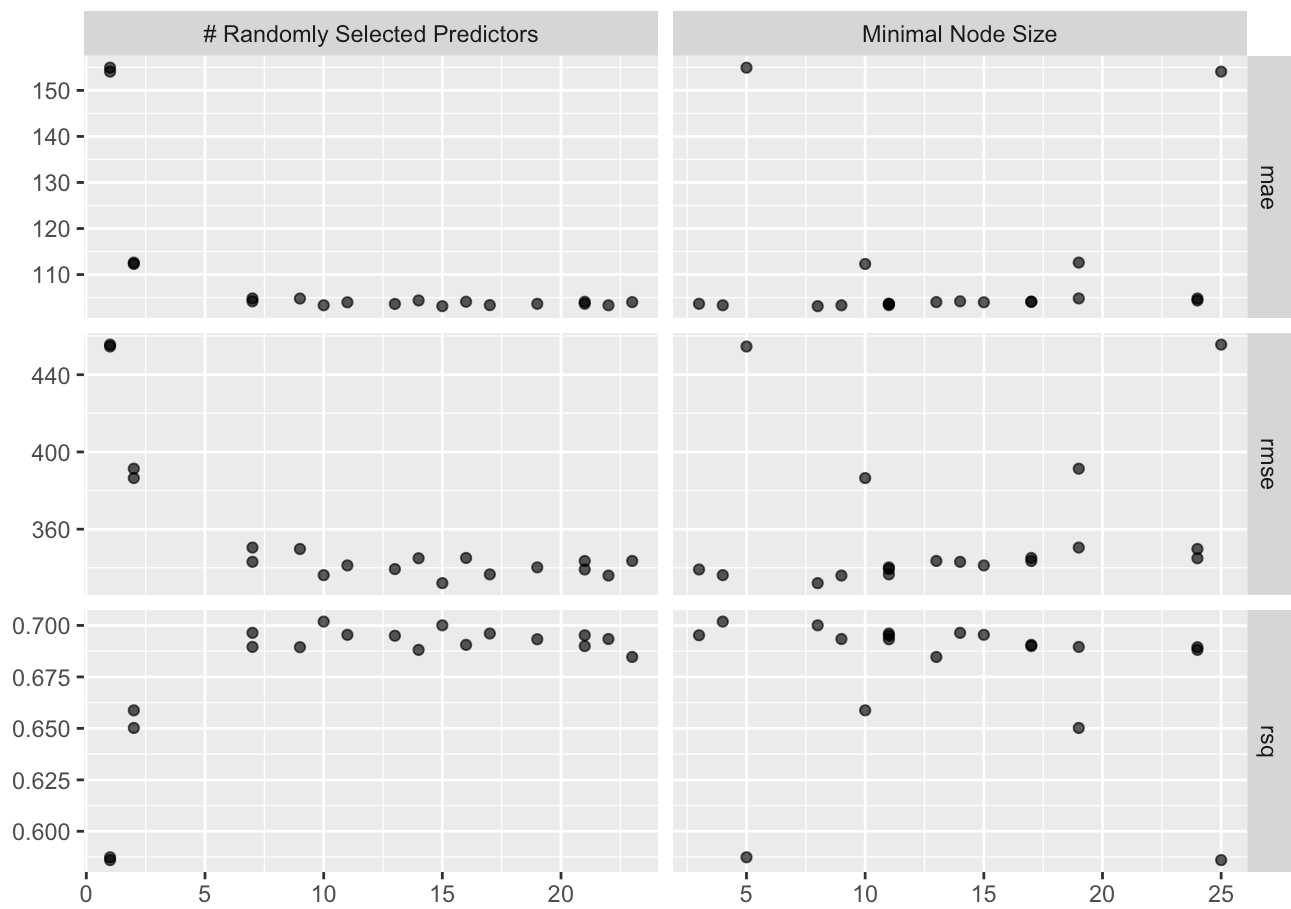
```
set.seed(88)
```

```
tune_results_rf_d = tune_grid(
  tune_rf_wf_d,
  resamples = folds_grouped,
  metrics = reg_metrics,
  grid = grid_random(rf_params_d, size = 20))
```

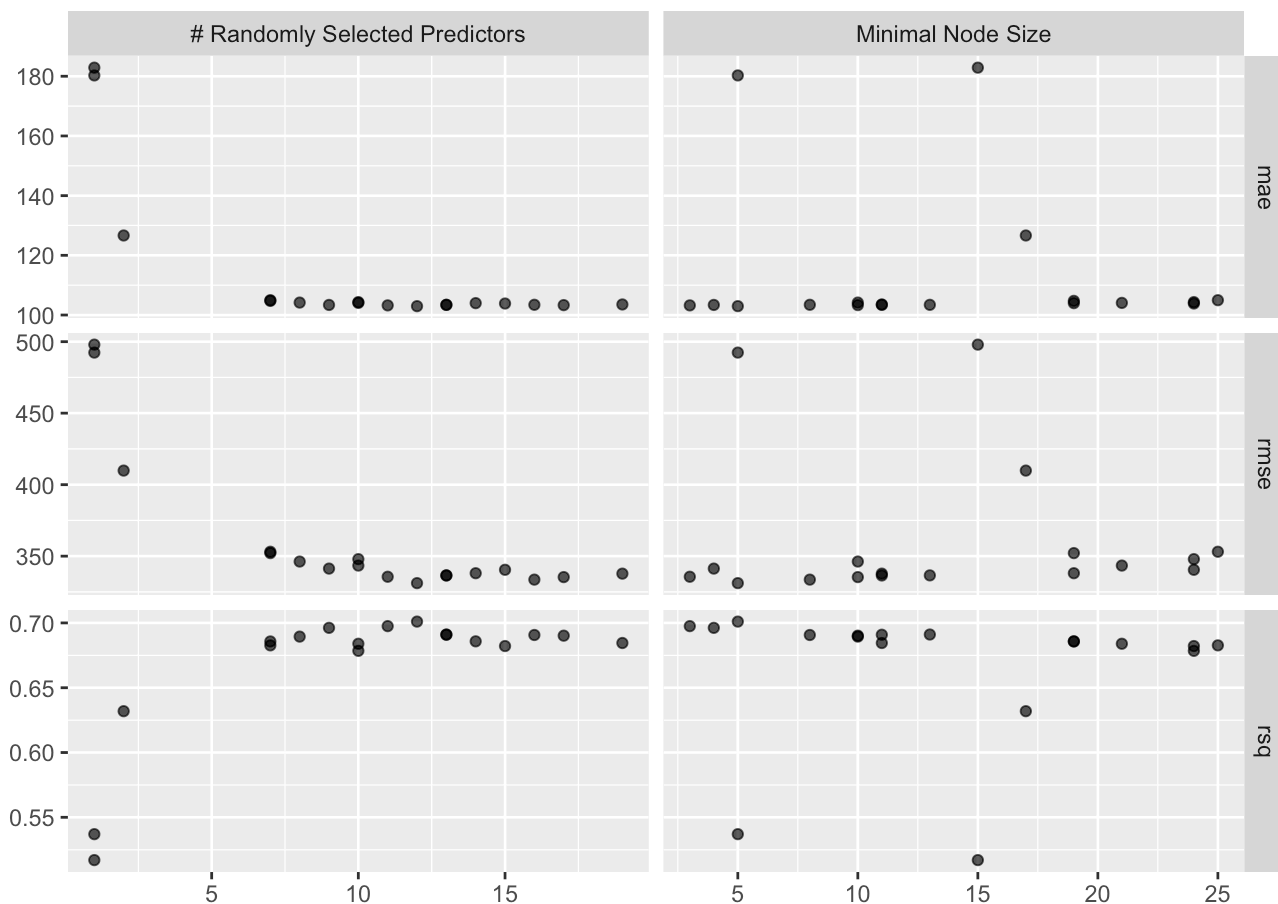
```
autoplot(tune_results_rf_a)
```



```
autoplot(tune_results_rf_c)
```



```
autoplot(tune_results_rf_d)
```



```
show_best(tune_results_rf_a, metric = "mae", n = 5)
```

# A tibble: 5 × 8

|   | mtry  | min_n | .metric | .estimator | mean  | n     | std_err | .config          |
|---|-------|-------|---------|------------|-------|-------|---------|------------------|
|   | <int> | <int> | <chr>   | <chr>      | <dbl> | <int> | <dbl>   | <chr>            |
| 1 | 10    | 4     | mae     | standard   | 102.  | 10    | 3.10    | pre0_mod08_post0 |
| 2 | 19    | 11    | mae     | standard   | 102.  | 10    | 2.97    | pre0_mod16_post0 |
| 3 | 15    | 8     | mae     | standard   | 102.  | 10    | 2.99    | pre0_mod13_post0 |
| 4 | 12    | 5     | mae     | standard   | 102.  | 10    | 3.07    | pre0_mod10_post0 |
| 5 | 13    | 11    | mae     | standard   | 102.  | 10    | 3.07    | pre0_mod11_post0 |

```
show_best(tune_results_rf_a, metric = "rmse", n = 5)
```

# A tibble: 5 × 8

|   | mtry  | min_n | .metric | .estimator | mean  | n     | std_err | .config          |
|---|-------|-------|---------|------------|-------|-------|---------|------------------|
|   | <int> | <int> | <chr>   | <chr>      | <dbl> | <int> | <dbl>   | <chr>            |
| 1 | 15    | 8     | rmse    | standard   | 313.  | 10    | 50.2    | pre0_mod13_post0 |
| 2 | 17    | 11    | rmse    | standard   | 315.  | 10    | 49.7    | pre0_mod15_post0 |
| 3 | 19    | 11    | rmse    | standard   | 316.  | 10    | 50.2    | pre0_mod16_post0 |
| 4 | 21    | 3     | rmse    | standard   | 317.  | 10    | 49.2    | pre0_mod17_post0 |
| 5 | 12    | 5     | rmse    | standard   | 318.  | 10    | 49.5    | pre0_mod10_post0 |

```
show_best(tune_results_rf_a, metric = "rsq", n = 5)
```

# A tibble: 5 × 8

|   | mtry  | min_n | .metric | .estimator | mean  | n     | std_err | .config          |
|---|-------|-------|---------|------------|-------|-------|---------|------------------|
|   | <int> | <int> | <chr>   | <chr>      | <dbl> | <int> | <dbl>   | <chr>            |
| 1 | 10    | 4     | rsq     | standard   | 0.730 | 10    | 0.0410  | pre0_mod08_post0 |
| 2 | 15    | 8     | rsq     | standard   | 0.729 | 10    | 0.0418  | pre0_mod13_post0 |
| 3 | 12    | 5     | rsq     | standard   | 0.728 | 10    | 0.0434  | pre0_mod10_post0 |
| 4 | 11    | 15    | rsq     | standard   | 0.724 | 10    | 0.0413  | pre0_mod09_post0 |
| 5 | 13    | 11    | rsq     | standard   | 0.724 | 10    | 0.0433  | pre0_mod11_post0 |

```
show_best(tune_results_rf_c, metric = "mae", n = 5)
```

# A tibble: 5 × 8

|   | mtry  | min_n | .metric | .estimator | mean  | n     | std_err | .config          |
|---|-------|-------|---------|------------|-------|-------|---------|------------------|
|   | <int> | <int> | <chr>   | <chr>      | <dbl> | <int> | <dbl>   | <chr>            |
| 1 | 15    | 8     | mae     | standard   | 103.  | 10    | 2.85    | pre0_mod12_post0 |
| 2 | 22    | 9     | mae     | standard   | 103.  | 10    | 3.04    | pre0_mod18_post0 |
| 3 | 10    | 4     | mae     | standard   | 103.  | 10    | 2.73    | pre0_mod08_post0 |
| 4 | 17    | 11    | mae     | standard   | 103.  | 10    | 2.95    | pre0_mod14_post0 |
| 5 | 13    | 11    | mae     | standard   | 104.  | 10    | 2.86    | pre0_mod10_post0 |

```
show_best(tune_results_rf_c, metric = "rmse", n = 5)
```

# A tibble: 5 × 8

|   | mtry  | min_n | .metric | .estimator | mean  | n     | std_err | .config          |
|---|-------|-------|---------|------------|-------|-------|---------|------------------|
|   | <int> | <int> | <chr>   | <chr>      | <dbl> | <int> | <dbl>   | <chr>            |
| 1 | 15    | 8     | rmse    | standard   | 332.  | 10    | 50.6    | pre0_mod12_post0 |
| 2 | 22    | 9     | rmse    | standard   | 336.  | 10    | 54.0    | pre0_mod18_post0 |
| 3 | 10    | 4     | rmse    | standard   | 336.  | 10    | 47.6    | pre0_mod08_post0 |
| 4 | 17    | 11    | rmse    | standard   | 337.  | 10    | 53.0    | pre0_mod14_post0 |
| 5 | 21    | 3     | rmse    | standard   | 339.  | 10    | 52.7    | pre0_mod16_post0 |

```
show_best(tune_results_rf_c, metric = "rsq", n = 5)
```

# A tibble: 5 × 8

|   | mtry  | min_n | .metric | .estimator | mean  | n     | std_err | .config          |
|---|-------|-------|---------|------------|-------|-------|---------|------------------|
|   | <int> | <int> | <chr>   | <chr>      | <dbl> | <int> | <dbl>   | <chr>            |
| 1 | 10    | 4     | rsq     | standard   | 0.702 | 10    | 0.0536  | pre0_mod08_post0 |
| 2 | 15    | 8     | rsq     | standard   | 0.700 | 10    | 0.0558  | pre0_mod12_post0 |
| 3 | 7     | 14    | rsq     | standard   | 0.696 | 10    | 0.0542  | pre0_mod05_post0 |
| 4 | 17    | 11    | rsq     | standard   | 0.696 | 10    | 0.0571  | pre0_mod14_post0 |
| 5 | 11    | 15    | rsq     | standard   | 0.695 | 10    | 0.0584  | pre0_mod09_post0 |

```
show_best(tune_results_rf_d, metric = "mae", n = 5)
```

# A tibble: 5 × 8

|   | mtry  | min_n | .metric | .estimator | mean  | n     | std_err | .config          |
|---|-------|-------|---------|------------|-------|-------|---------|------------------|
|   | <int> | <int> | <chr>   | <chr>      | <dbl> | <int> | <dbl>   | <chr>            |
| 1 | 12    | 5     | mae     | standard   | 103.  | 10    | 3.30    | pre0_mod11_post0 |
| 2 | 11    | 3     | mae     | standard   | 103.  | 10    | 3.25    | pre0_mod10_post0 |
| 3 | 17    | 10    | mae     | standard   | 103.  | 10    | 3.12    | pre0_mod17_post0 |

|   |    |    |     |          |      |    |      |                  |
|---|----|----|-----|----------|------|----|------|------------------|
| 4 | 9  | 4  | mae | standard | 103. | 10 | 3.40 | pre0_mod07_post0 |
| 5 | 13 | 11 | mae | standard | 103. | 10 | 3.22 | pre0_mod12_post0 |

```
show_best(tune_results_rf_d, metric = "rmse", n = 5)
```

# A tibble: 5 × 8

|   | mtry  | min_n | .metric | .estimator | mean  | n     | std_err | .config          |
|---|-------|-------|---------|------------|-------|-------|---------|------------------|
|   | <int> | <int> | <chr>   | <chr>      | <dbl> | <int> | <dbl>   | <chr>            |
| 1 | 12    | 5     | rmse    | standard   | 331.  | 10    | 56.3    | pre0_mod11_post0 |
| 2 | 16    | 8     | rmse    | standard   | 334.  | 10    | 56.0    | pre0_mod16_post0 |
| 3 | 17    | 10    | rmse    | standard   | 335.  | 10    | 57.3    | pre0_mod17_post0 |
| 4 | 11    | 3     | rmse    | standard   | 336.  | 10    | 56.2    | pre0_mod10_post0 |
| 5 | 13    | 11    | rmse    | standard   | 336.  | 10    | 57.5    | pre0_mod12_post0 |

```
show_best(tune_results_rf_d, metric = "rsq", n = 5)
```

# A tibble: 5 × 8

|   | mtry  | min_n | .metric | .estimator | mean  | n     | std_err | .config          |
|---|-------|-------|---------|------------|-------|-------|---------|------------------|
|   | <int> | <int> | <chr>   | <chr>      | <dbl> | <int> | <dbl>   | <chr>            |
| 1 | 12    | 5     | rsq     | standard   | 0.701 | 10    | 0.0391  | pre0_mod11_post0 |
| 2 | 11    | 3     | rsq     | standard   | 0.698 | 10    | 0.0395  | pre0_mod10_post0 |
| 3 | 9     | 4     | rsq     | standard   | 0.696 | 10    | 0.0375  | pre0_mod07_post0 |
| 4 | 13    | 13    | rsq     | standard   | 0.691 | 10    | 0.0413  | pre0_mod13_post0 |
| 5 | 13    | 11    | rsq     | standard   | 0.691 | 10    | 0.0406  | pre0_mod12_post0 |

Random Forests seem really strong!!! That's good. Let's investigate some more.

```
best_rf_a = select_best(tune_results_rf_a, metric = "mae")
best_rf_c = select_best(tune_results_rf_c, metric = "mae")
best_rf_d = select_best(tune_results_rf_d, metric = "mae")
```

```
collect_metrics(tune_results_rf_a) |>
  semi_join(best_rf_a, by = names(best_rf_a)) |>
  mutate(model = "RF A")
```

# A tibble: 3 × 9

|   | mtry  | min_n | .metric | .estimator | mean  | n     | std_err | .config          | model |
|---|-------|-------|---------|------------|-------|-------|---------|------------------|-------|
|   | <int> | <int> | <chr>   | <chr>      | <dbl> | <int> | <dbl>   | <chr>            | <chr> |
| 1 | 10    | 4     | mae     | standard   | 102.  | 10    | 3.10    | pre0_mod08_post0 | RF A  |
| 2 | 10    | 4     | rmse    | standard   | 319.  | 10    | 51.3    | pre0_mod08_post0 | RF A  |
| 3 | 10    | 4     | rsq     | standard   | 0.730 | 10    | 0.0410  | pre0_mod08_post0 | RF A  |

```
rf_best_metrics = bind_rows(
  collect_metrics(tune_results_rf_a) |>
    semi_join(best_rf_a, by = names(best_rf_a)) |>
    mutate(model = "RF A"),
  collect_metrics(tune_results_rf_c) |>
    semi_join(best_rf_c, by = names(best_rf_c)) |>
    mutate(model = "RF C"),
  collect_metrics(tune_results_rf_d) |>
    semi_join(best_rf_d, by = names(best_rf_d)) |>
```

```
mutate(model = "RF D"))
```

```
rf_best_metrics
```

```
# A tibble: 9 × 9
```

|   | mtry  | min_n | .metric | .estimator | mean  | n     | std_err | .config          | model |
|---|-------|-------|---------|------------|-------|-------|---------|------------------|-------|
|   | <int> | <int> | <chr>   | <chr>      | <dbl> | <int> | <dbl>   | <chr>            | <chr> |
| 1 | 10    | 4     | mae     | standard   | 102.  | 10    | 3.10    | pre0_mod08_post0 | RF A  |
| 2 | 10    | 4     | rmse    | standard   | 319.  | 10    | 51.3    | pre0_mod08_post0 | RF A  |
| 3 | 10    | 4     | rsq     | standard   | 0.730 | 10    | 0.0410  | pre0_mod08_post0 | RF A  |
| 4 | 15    | 8     | mae     | standard   | 103.  | 10    | 2.85    | pre0_mod12_post0 | RF C  |
| 5 | 15    | 8     | rmse    | standard   | 332.  | 10    | 50.6    | pre0_mod12_post0 | RF C  |
| 6 | 15    | 8     | rsq     | standard   | 0.700 | 10    | 0.0558  | pre0_mod12_post0 | RF C  |
| 7 | 12    | 5     | mae     | standard   | 103.  | 10    | 3.30    | pre0_mod11_post0 | RF D  |
| 8 | 12    | 5     | rmse    | standard   | 331.  | 10    | 56.3    | pre0_mod11_post0 | RF D  |
| 9 | 12    | 5     | rsq     | standard   | 0.701 | 10    | 0.0391  | pre0_mod11_post0 | RF D  |

Let's try SVM now (at least a radial version of it, already have a linear/non-linear, lets get radial)

```
set.seed(88)
```

```
svm_spec = svm_rbf(
  mode = "regression",
  cost = tune(),
  rbf_sigma = tune()) |>
  set_engine("kernlab")
svm_wf_a = workflow() |>
  add_recipe(recipe_a_reg) |>
  add_model(svm_spec)
svm_params_a = extract_parameter_set_dials(svm_wf_a)

tune_results_svm_a = tune_grid(
  svm_wf_a,
  resamples = folds_grouped,
  metrics = reg_metrics,
  grid = grid_random(svm_params_a, size = 10))
```

```
show_best(tune_results_svm_a, metric = "mae", n = 5)
```

```
# A tibble: 5 × 8
```

|   | cost  | rbf_sigma   | .metric | .estimator | mean  | n     | std_err | .config          |
|---|-------|-------------|---------|------------|-------|-------|---------|------------------|
|   | <dbl> | <dbl>       | <chr>   | <chr>      | <dbl> | <int> | <dbl>   | <chr>            |
| 1 | 25.1  | 0.00425     | mae     | standard   | 101.  | 10    | 2.62    | pre0_mod08_post0 |
| 2 | 1.11  | 0.00300     | mae     | standard   | 108.  | 10    | 5.13    | pre0_mod05_post0 |
| 3 | 2.64  | 0.0000850   | mae     | standard   | 116.  | 10    | 5.41    | pre0_mod07_post0 |
| 4 | 2.17  | 0.0000339   | mae     | standard   | 117.  | 10    | 5.50    | pre0_mod06_post0 |
| 5 | 31.8  | 0.000000202 | mae     | standard   | 134.  | 10    | 6.19    | pre0_mod10_post0 |

```
best_svm_a = select_best(
  tune_results_svm_a,
  metric = "mae")
```

```
collect_metrics(tune_results_svm_a) |>
  semi_join(
    best_svm_a,
    by = names(best_svm_a))
```

# A tibble: 3 × 8

|   | cost  | rbf_sigma | .metric | .estimator | mean  | n     | std_err | .config          |
|---|-------|-----------|---------|------------|-------|-------|---------|------------------|
|   | <dbl> | <dbl>     | <chr>   | <chr>      | <dbl> | <int> | <dbl>   | <chr>            |
| 1 | 25.1  | 0.00425   | mae     | standard   | 101.  | 10    | 2.62    | pre0_mod08_post0 |
| 2 | 25.1  | 0.00425   | rmse    | standard   | 306.  | 10    | 43.2    | pre0_mod08_post0 |
| 3 | 25.1  | 0.00425   | rsq     | standard   | 0.765 | 10    | 0.0380  | pre0_mod08_post0 |

```
final_svm_wf = svm_wf_a |>
  finalize_workflow(best_svm_a)

final_svm_fit = final_svm_wf |>
  fit(data = train_data)
```

## Predict on testing data

```
final_linreg_a = lr_wf_a |>
  fit(data = train_data)
final_xgboost_a = tune_xgboost_wf_a |>
  finalize_workflow(best_xgboost_a) |>
  fit(data = train_data)
final_rf_a = tune_rf_wf_a |>
  finalize_workflow(best_rf_a) |>
  fit(data = train_data)
final_svm_a = svm_wf_a |>
  finalize_workflow(best_svm_a) |>
  fit(data = train_data)

linreg_test = final_linreg_a |>
  augment(new_data = test_data)
xgboost_test = final_xgboost_a |>
  augment(new_data = test_data)
rf_test = final_rf_a |>
  augment(new_data = test_data)
svm_test = final_svm_a |>
  augment(new_data = test_data)
```

Now we calculate our metrics

```
test_results = bind_rows(
  linreg_test |>
    reg_metrics(truth = private_k12, estimate = .pred) |>
    mutate(model = "Linear Regression"),
  xgboost_test |>
    reg_metrics(truth = private_k12, estimate = .pred) |>
```

```

mutate(model = "XGBoost"),
rf_test |>
  reg_metrics(truth = private_k12, estimate = .pred) |>
  mutate(model = "Random Forest"),
svm_test |>
  reg_metrics(truth = private_k12, estimate = .pred) |>
  mutate(model = "RBF SVM")

```

test\_results

# A tibble: 12 × 4

|    | .metric | .estimator | .estimate | model             |
|----|---------|------------|-----------|-------------------|
|    | <chr>   | <chr>      | <dbl>     | <chr>             |
| 1  | rmse    | standard   | 518.      | Linear Regression |
| 2  | rsq     | standard   | 0.640     | Linear Regression |
| 3  | mae     | standard   | 198.      | Linear Regression |
| 4  | rmse    | standard   | 248.      | XGBoost           |
| 5  | rsq     | standard   | 0.921     | XGBoost           |
| 6  | mae     | standard   | 102.      | XGBoost           |
| 7  | rmse    | standard   | 233.      | Random Forest     |
| 8  | rsq     | standard   | 0.942     | Random Forest     |
| 9  | mae     | standard   | 92.1      | Random Forest     |
| 10 | rmse    | standard   | 308.      | RBF SVM           |
| 11 | rsq     | standard   | 0.908     | RBF SVM           |
| 12 | mae     | standard   | 106.      | RBF SVM           |

```

test_results_table = test_results |>
  dplyr::select(model, .metric, .estimate) |>
  pivot_wider(names_from = .metric, values_from = .estimate) |>
  arrange(mae)

```

test\_results\_table

# A tibble: 4 × 4

|   | model             | rmse  | rsq   | mae   |
|---|-------------------|-------|-------|-------|
|   | <chr>             | <dbl> | <dbl> | <dbl> |
| 1 | Random Forest     | 233.  | 0.942 | 92.1  |
| 2 | XGBoost           | 248.  | 0.921 | 102.  |
| 3 | RBF SVM           | 308.  | 0.908 | 106.  |
| 4 | Linear Regression | 518.  | 0.640 | 198.  |

```

rf_test_errors = rf_test |>
  mutate(
    error = .pred - private_k12,
    abs_error = abs(error))

rf_test_errors |>
  summarize(
    mean_absolute_error = mean(abs_error),
    median_absolute_error = median(abs_error),
    q25_absolute_error = quantile(abs_error, 0.25),

```

```
q75_absolute_error = quantile(abs_error, 0.75),
q90_absolute_error = quantile(abs_error, 0.90))
```

# A tibble: 1 × 5

```
mean_absolute_error median_absolute_error q25_absolute_error
      <dbl>              <dbl>              <dbl>
1      92.1             42.7             17.0
# i 2 more variables: q75_absolute_error <dbl>, q90_absolute_error <dbl>
```

```
rf_test_errors = rf_test_errors |>
  mutate(
    enrollment_group = case_when(
      private_k12 < 50 ~ "Under 50",
      private_k12 < 100 ~ "50-99",
      private_k12 < 250 ~ "100-249",
      private_k12 < 500 ~ "250-499",
      private_k12 < 1000 ~ "500-999",
      TRUE ~ "1000+")
  )

rf_test_errors |>
  group_by(enrollment_group) |>
  summarize(
    n = n(),
    mean_actual = mean(private_k12),
    mae = mean(abs_error),
    median_abs_error = median(abs_error),
    rmse = sqrt(mean(error^2))
  )
```

# A tibble: 6 × 6

|   | enrollment_group | n     | mean_actual | mae   | median_abs_error | rmse  |
|---|------------------|-------|-------------|-------|------------------|-------|
|   | <chr>            | <int> | <dbl>       | <dbl> | <dbl>            | <dbl> |
| 1 | 100-249          | 1295  | 162.        | 74.7  | 60.6             | 102.  |
| 2 | 1000+            | 260   | 2070.       | 546.  | 376.             | 978.  |
| 3 | 250-499          | 841   | 354.        | 122.  | 103.             | 159.  |
| 4 | 50-99            | 888   | 72.3        | 43.6  | 31.8             | 63.8  |
| 5 | 500-999          | 629   | 700.        | 170.  | 142.             | 219.  |
| 6 | Under 50         | 1993  | 18.2        | 28.5  | 18.2             | 45.0  |

This looks really good once zips get big but not as good for smaller ones. It looks like there are likely some predicted negative values, which can't be possible in real life.

```
rf_test |> summarize(
  negative_predictions = sum(.pred < 0),
  pct_negative = mean(.pred < 0) * 100,
  minimum_prediction = min(.pred))
```

# A tibble: 1 × 3

|   | negative_predictions | pct_negative | minimum_prediction |
|---|----------------------|--------------|--------------------|
|   | <int>                | <dbl>        | <dbl>              |
| 1 | 0                    | 0            | 2.20               |

# A tibble: 5,906 × 158

|    | .pred | year  | ZIP   | median_hh_income | median_home_value | male_age_0_4 |
|----|-------|-------|-------|------------------|-------------------|--------------|
|    | <dbl> | <dbl> | <chr> | <dbl>            | <dbl>             | <dbl>        |
| 1  | 172.  | 2024  | 01001 | 75342            | 295700            | 426          |
| 2  | 421.  | 2024  | 01002 | 70818            | 481100            | 257          |
| 3  | 83.7  | 2024  | 01005 | 111148           | 392600            | 112          |
| 4  | 239.  | 2024  | 01007 | 108125           | 371400            | 337          |
| 5  | 24.9  | 2024  | 01008 | 97554            | 295200            | 32           |
| 6  | 60.6  | 2024  | 01010 | 111250           | 389100            | 61           |
| 7  | 33.5  | 2024  | 01011 | 89299            | 291400            | 14           |
| 8  | 328.  | 2024  | 01013 | 59759            | 271900            | 722          |
| 9  | 393.  | 2024  | 01020 | 66009            | 280900            | 634          |
| 10 | 19.5  | 2024  | 01022 | 51373            | 193600            | 71           |

# i 5,896 more rows

# i 152 more variables: male\_age\_5\_9 <dbl>, male\_age\_10\_14 <dbl>,  
 # male\_age\_15\_17 <dbl>, female\_age\_0\_4 <dbl>, female\_age\_5\_9 <dbl>,  
 # female\_age\_10\_14 <dbl>, female\_age\_15\_17 <dbl>, age\_0\_4 <dbl>,  
 # age\_5\_9 <dbl>, age\_10\_14 <dbl>, age\_15\_17 <dbl>,  
 # male\_school\_age\_population <dbl>, female\_school\_age\_population <dbl>,  
 # school\_age\_population <dbl>, pct\_school\_age\_male <dbl>, ...

```
set.seed(88)
```

```
train_data_rate = train_data |>
  mutate(private_rate = private_k12 / school_age_population)
```

```
test_data_rate = test_data |>
  mutate(private_rate = private_k12 / school_age_population)
```

```
folds_grouped_rate = group_vfold_cv(
  train_data_rate, group = ZIP, v = 10)
```

```
recipe_a_rate = recipe(
  private_rate ~
    median_hh_income_inflation_adjusted + median_home_value_inflation_adjusted +
    school_age_population + pct_adults_hs_plus + pct_adults_bachelors_plus +
    pct_adults_grad_plus + total_households + pct_households_with_children +
    age_0_4 + state,
  data = train_data_rate) |>
  step_dummy(all_nominal_predictors()) |>
  step_zv(all_predictors())
```

```
rf_rate_spec = rand_forest(
  mode = "regression",
  mtry = tune(),
  min_n = tune(),
  trees = 300) |>
  set_engine("ranger", importance = "impurity")
```

```
rf_rate_wf = workflow() |>
```

```

add_recipe(recipe_a_rate) |>
add_model(rf_rate_spec)
rf_rate_prepped = prep(recipe_a_rate)
rf_rate_num_predictors = rf_rate_prepped |>
  bake(new_data = NULL) |>
  select(-private_rate) |>
  ncol()
rf_rate_params = extract_parameter_set_dials(rf_rate_wf) |>
  update(
    mtry = mtry(c(1, rf_rate_num_predictors)),
    min_n = min_n(c(2, 25)))

```

Now we tune based on this new data

```

set.seed(88)

tune_results_rf_rate = tune_grid(
  rf_rate_wf,
  resamples = folds_grouped_rate,
  metrics = reg_metrics,
  grid = grid_random(
    rf_rate_params,
    size = 20))

show_best(tune_results_rf_rate, metric = "mae", n = 5)

```

# A tibble: 5 × 8

|   | mtry  | min_n | .metric | .estimator | mean   | n     | std_err  | .config          |
|---|-------|-------|---------|------------|--------|-------|----------|------------------|
|   | <int> | <int> | <chr>   | <chr>      | <dbl>  | <int> | <dbl>    | <chr>            |
| 1 | 7     | 19    | mae     | standard   | 0.0645 | 10    | 0.000772 | pre0_mod06_post0 |
| 2 | 9     | 24    | mae     | standard   | 0.0646 | 10    | 0.000739 | pre0_mod07_post0 |
| 3 | 7     | 14    | mae     | standard   | 0.0646 | 10    | 0.000756 | pre0_mod05_post0 |
| 4 | 14    | 24    | mae     | standard   | 0.0649 | 10    | 0.000763 | pre0_mod12_post0 |
| 5 | 11    | 15    | mae     | standard   | 0.0649 | 10    | 0.000767 | pre0_mod09_post0 |

```

best_rf_rate = select_best(tune_results_rf_rate, metric = "mae")
best_rf_rate

```

# A tibble: 1 × 3

|   | mtry  | min_n | .config          |
|---|-------|-------|------------------|
|   | <int> | <int> | <chr>            |
| 1 | 7     | 19    | pre0_mod06_post0 |

```

collect_metrics(tune_results_rf_rate) |>
  semi_join(
    best_rf_rate,
    by = names(best_rf_rate))

```

# A tibble: 3 × 8

|  | mtry  | min_n | .metric | .estimator | mean  | n     | std_err | .config |
|--|-------|-------|---------|------------|-------|-------|---------|---------|
|  | <int> | <int> | <chr>   | <chr>      | <dbl> | <int> | <dbl>   | <chr>   |

|   |   |    |      |          |        |    |          |                  |
|---|---|----|------|----------|--------|----|----------|------------------|
| 1 | 7 | 19 | mae  | standard | 0.0645 | 10 | 0.000772 | pre0_mod06_post0 |
| 2 | 7 | 19 | rmse | standard | 0.0933 | 10 | 0.00172  | pre0_mod06_post0 |
| 3 | 7 | 19 | rsq  | standard | 0.216  | 10 | 0.0142   | pre0_mod06_post0 |

```
final_rf_rate_wf = rf_rate_wf |>
  finalize_workflow(best_rf_rate)

final_rf_rate = final_rf_rate_wf |>
  fit(data = train_data_rate)
```

Now we can predict on 2024 data

```
rf_rate_test = final_rf_rate |>
  augment(new_data = test_data_rate) |>
  mutate(.pred_count = .pred * school_age_population)
rf_rate_test_metrics = rf_rate_test |>
  reg_metrics(truth = private_k12, estimate = .pred_count)
rf_rate_test_metrics
```

```
# A tibble: 3 × 3
  .metric .estimator .estimate
  <chr>   <chr>       <dbl>
1 rmse    standard      229.
2 rsq     standard       0.931
3 mae     standard       95.1
```

```
rf_rate_comparison = bind_rows(
  rf_test |>
    reg_metrics(truth = private_k12, estimate = .pred) |>
    mutate(model = "RF Raw Count"),
  rf_rate_test |>
    reg_metrics(truth = private_k12, estimate = .pred_count) |>
    mutate(model = "RF Rate")) |>
  dplyr::select(model, .metric, .estimate) |>
  pivot_wider(
    names_from = .metric,
    values_from = .estimate)

rf_rate_comparison
```

```
# A tibble: 2 × 4
  model      rmse  rsq  mae
  <chr>    <dbl> <dbl> <dbl>
1 RF Raw Count  233. 0.942 92.1
2 RF Rate      229. 0.931 95.1
```

This should really help our tails, as the outliers from the previous model were really off.

```
rf_rate_errors = rf_rate_test |>
  mutate(
    error = .pred_count - private_k12,
```

```

    abs_error = abs(error))

rf_rate_errors |>
  summarize(
    mean_absolute_error = mean(abs_error),
    median_absolute_error = median(abs_error),
    q25_absolute_error = quantile(abs_error, 0.25),
    q75_absolute_error = quantile(abs_error, 0.75),
    q90_absolute_error = quantile(abs_error, 0.90))

# A tibble: 1 × 5
  mean_absolute_error median_absolute_error q25_absolute_error
      <dbl>              <dbl>              <dbl>
1         95.1             42.0             16.5
# i 2 more variables: q75_absolute_error <dbl>, q90_absolute_error <dbl>

```

It does better with the outliers, hooray! The Random Forest capturing rate of private school enrollment rather than raw count is strongest.

Stop our clustering!

```
parallel::stopCluster(cl)
```

```
dataframe = read_csv("northeast_census_data_gender.csv")
```

Rows: 49522 Columns: 157

— Column specification —

Delimiter: ","

chr (3): ZIP, zip\_place\_name, state

dbl (152): year, median\_hh\_income, median\_home\_value, male\_age\_0\_4, male\_age...

lgl (2): complete\_history, high\_private\_share

- i Use `spec()` to retrieve the full column specification for this data.
- i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```

ne = ne |>
  mutate(actual_private_rate = private_k12 / school_age_population)

rf_rate_features = rf_rate_test |>
  transmute(ZIP, year, projected_private_rate = .pred, projected_private_k12 = .pred_count)

ne = ne |>
  left_join(rf_rate_features, by = c("ZIP", "year"))

```

```
write_csv(ne, 'northeast_data_modeling_final.csv')
```